

[IME PREZIME]

Razvoj aplikacije za rutiranje teretnih vozila prilagođene fizičkim ograničenjima vozila i terena

Diplomski rad
- Osnovne akademske studije -

Ključna dokumentacijska informacija

Redni broj, RBR:

Identifikacioni broj, IBR:

Tip dokumentacije, TD: Monografska dokumentacija

Tip zapisa, TZ: Tekstualni štampani materijal

Vrsta rada, VR: Diplomski rad

Autor, AU: [IME PREZIME]

Mentor, MN: [TITULA I IME MENTORA]

Naslov rada, NR: Razvoj aplikacije za rutiranje teretnih vozila prilagođene fizičkim ograničenjima vozila i terena

Jezik publikacije, JP: Srpski / latinica

Jezik izvoda, JI: Srpski

Zemlja publikovanja, ZP: Republika Srbija

Uže geografsko područje, Vojvodina

UGP:

Godina, GO: 2026

Izdavač, IZ: Autorski reprint

Mesto i adresa, MA: Novi Sad, Trg Dositeja Obradovića 6

Fizički opis rada, FO: 10/[BROJ]/17/[BROJ]/[BROJ]/3/0

Naučna oblast, NO: Elektrotehnika i računarstvo

Naučna disciplina, ND: Primenjeno softversko inženjerstvo

**Predmetna odrednica-
/Ključne reči, PO:** Rutiranje teretnih vozila, OpenStreetMap, Valhalla, distribuirani sistemi, mobilna aplikacija, prilagođena funkcija cene rute

UDK

Čuva se, ČU: U biblioteci Fakulteta tehničkih nauka, Novi Sad

Važna napomena, VN:

Izvod, IZ:

U ovom radu opisan je sistem za rutiranje teretnih vozila koji uzima u obzir fizička ograničenja vozila (visina, širina, dužina, masa, osovinsko opterećenje, prevoz opasnog tereta) i podatke o putnoj mreži preuzete iz OpenStreetMap projekta. Sistem se sastoji od Go backend servisa, Valhalla routing engine-a sa truck costing modelom, PostgreSQL/PostGIS baze podataka, RabbitMQ mehanizma za asinhronu obradu i Flutter mobilne aplikacije za vozače i dispečere. Nad Valhalla alternativama implementiran je sopstveni modul za rangiranje ruta prema riziku i preferencama vozača, kao i mehanizam koji vozaču objašnjava zašto predložena ruta odstupa od geometrijski najkraćeg puta. Za potrebe algoritamske evaluacije implementirana je sopstvena Dijkstra/A* pretraga nad ograničenim podgrafom putne mreže.

Datum prihvatanja teme,**DP:****Datum odbrane, DO:****Članovi komisije, KO:**

Predsednik:

Član:

Član, mentor: [TITULA I IME MENTORA]

Key words documentation

Document type, DT:	Monographic publication
Contents code, CC:	Bachelor thesis
Author, AU:	[NAME SURNAME]
Mentor, MN:	[MENTOR TITLE AND NAME]
Title, TI:	Development of a heavy vehicle routing application adapted to the physical constraints of the vehicle and terrain
Language of text, LT:	Serbian
Country of publication, CP:	Republic of Serbia
Publication year, PY:	2026
Scientific field, SF:	Electrical and Computer Engineering
Subject/Key words, S/KW:	Heavy vehicle routing, OpenStreetMap, Valhalla, distributed systems, mobile application, custom route cost function
Abstract, AB:	This thesis describes a heavy vehicle routing system that accounts for the vehicle's physical constraints using road network data from OpenStreetMap, combining the Valhalla routing engine's truck costing model with a custom risk-ranking and route-explanation layer, a Go backend, RabbitMQ-based asynchronous processing, and a Flutter mobile application for drivers and dispatchers.

Zadatak za izradu diplomskog (Bachelor) rada

Vrsta studija: Osnovne akademske studije

Studijski program: [STUDIJSKI PROGRAM]

Student: [IME PREZIME] **Broj indeksa:** [BROJ INDEKSA]

Oblast: Elektrotehničko i računarsko inženjerstvo

Mentor: [TITULA I IME MENTORA]

Tekst zadatka:

- Proučiti postojeća komercijalna i otvorena rešenja za rutiranje teretnih vozila i principe rada distribuiranih sistema.
- Proučiti podatkovni model OpenStreetMap projekta u delu koji se odnosi na ograničenja za teretni saobraćaj i pripremiti geografske podatke za teritoriju Republike Srbije.
- Implementirati backend servis koji generiše, rangira sopstvenom funkcijom rizika i objašnjava predloženu rutu za dato vozilo.
- Implementirati, u svrhu algoritamske evaluacije, sopstvenu pretragu najkraćeg puta (Dijkstra/A*) nad ograničenim podgrafom putne mreže.
- Implementirati distribuiranu komunikaciju sistema i mobilnu aplikaciju za vozače i dispečere.
- Testirati i evaluirati implementirano rešenje i izvesti najvažnije zaključke.

Izjava o nepostojanju sukoba interesa

Izjavljujem da nisam u sukobu interesa u odnosu mentor – kandidat i da nisam član porodice, povezano lice, odnosno da nisam zavisana od mentora/kandidata, da ne postoje okolnosti koje bi mogle da utiču na moju nepristrasnost, niti da stičem bilo kakve koristi ili pogodnosti bilo pozitivnim ili negativnim ishodom.

U Novom Sadu, dana _____

Mentor

Kandidat

„The map is not the territory.“

— Alfred Korzybski

Sadržaj

1	Uvod	1
1.1	Motivacija	2
1.2	Ciljevi i obim rada	2
1.3	Struktura rada	2
2	Pregled postojećih rešenja i teorijske osnove	4
2.1	Komercijalna i otvorena rešenja za rutiranje teretnih vozila	4
2.2	Distribuirani sistemi — osnovni pojmovi	5
2.3	Grafovi i algoritmi najkraćeg puta	6
2.4	OpenStreetMap podatkovni model i oznake za teretni saobraćaj	7
3	Korišćene tehnologije	9
3.1	Go	9
3.2	Flutter i Dart	9
3.3	Valhalla	9
3.4	PostgreSQL i PostGIS	10
3.5	RabbitMQ	10
3.6	WebSocket	10
3.7	Docker i docker-compose	10
3.8	Ostale biblioteke i servisi	11
4	Arhitektura sistema	12
4.1	Pregled arhitekture i tok podataka	12
4.2	Kontejnerizacija i orkestracija servisa	13
4.3	Sigurnost i autentifikacija	14
5	Priprema geografskih podataka — OSM ekstrakcija za teretna vozila u Srbiji	16
5.1	Izvor podataka i osmium filter	16
5.2	Izgradnja Valhalla grafa	17
5.3	Ekstrakcija podataka o odmaralištima	17
6	Algoritam rutiranja prilagođen vozilu	18
6.1	Valhalla truck costing	18
6.2	Prilagođena funkcija rizika (scoring)	20
6.3	Objašnjenje predložene rute — slučaj Novi Banovci	21
6.4	Sopstvena implementacija pretrage puta nad ograničenim podgrafom	23

7	Backend servis i model podataka	27
7.1	REST API	27
7.2	Model podataka	29
7.3	Uloge vozač/dispečer	31
7.4	Asinhrona obrada — RabbitMQ i modul pauza vozača	31
7.5	Real-time komunikacija — WebSocket gateway	33
8	Mobilna aplikacija	35
8.1	Tok korisnika — vozač	35
8.2	Tok korisnika — dispečer	36
8.3	Arhitektura mobilne aplikacije	36
9	Testiranje i evaluacija	38
9.1	Automatizovano testiranje backend-a	38
9.2	Evaluacija sopstvenog algoritma nad realnim podacima	39
9.3	Uticaj prilagođene funkcije rizika — studija slučaja	40
9.4	Ograničenja identifikovana testiranjem	41
10	Zaključak i budući rad	42
10.1	Doprinosi rada	42
10.2	Ograničenja rada	43
10.3	Pravci budućeg rada	43
	Literatura	45
	Spisak skraćenica	47
	Spisak slika	47
	Spisak tabela	48
	Spisak listinga	50
	Biografija	52

1 Uvod

Transport robe teškim teretnim vozilima podleže ograničenjima koja ne postoje kod putničkih automobila: visina vozila mora biti manja od visine svakog nadvožnjaka i tunela na ruti, ukupna masa i osovinsko opterećenje moraju biti u skladu sa nosivošću mostova, širina i dužina vozila ograničavaju kretanje kroz uža naselja i oštre krivine, a vozila koja prevoze opasan teret moraju zaobilaziti određene deonice (tunele, naseljena mesta) po propisu. Standardne aplikacije za navigaciju namenjene putničkim vozilima (Google Maps, Waze i slične) te podatke ne uzimaju u obzir, pa ruta koju predlažu vozaču teretnog vozila može biti fizički neprohodna ili čak zakonski nedozvoljena za dato vozilo. Sa druge strane, postojeća komercijalna rešenja specijalizovana za teretni saobraćaj (npr. PTV navigator, Google Maps za dostavu, TomTom Truck) su zatvorena, plaćena i ne omogućavaju uvid u to *zašto* je određena ruta izabrana, što otežava vozaču da proceni da li je predlog razuman.

Ovaj rad opisuje razvoj sistema za rutiranje teretnih vozila koji rešava taj problem korišćenjem otvorenih geografskih podataka projekta OpenStreetMap (OSM) i otvorenog routing engine-a Valhalla, koji već ima ugrađen model cene rute specifičan za teretna vozila (*truck costing*), sposoban da isključi deonice koje ne zadovoljavaju ograničenja visine, mase, širine ili prevoza opasnog tereta datog vozila. Na tu osnovu je dodat sopstveni sloj koji:

1. rangira alternativne rute koje Valhalla vrati prema riziku i preferencama vozača (stil vožnje, osetljivost tereta, potrošnja goriva, favorizovane lokacije na mapi), pošto sam Valhalla to ne radi;
2. vozaču objašnjava **zašto** je predložena ruta odstupila od geometrijski najkraćeg puta, umesto da mu samo prikaže krajnji rezultat;
3. omogućava, kao samostalnu algoritamsku celinu odvojenu od produkcionog puta, sopstvenu implementaciju Dijkstra i A* pretrage najkraćeg puta nad ograničenim podgrafom putne mreže, radi demonstracije i evaluacije principa na kojima Valhalla interno počiva.

Sistem je zamišljen ne samo kao alat za planiranje jedne rute, već kao kompletna, iako po obimu ograničena, distribuirana aplikacija za rad voznog parka: pored planiranja i praćenja pojedinačne ture, uveden je i model dispečera koji upravlja više vozača i vozila, mehanizam ponude i prihvatanja ture između dispečera i vozača, chat u realnom vremenu, praćenje pozicije vozila uživo preko WebSocket veze, dnevnik događaja tokom vožnje i jednostavan mehanizam predloga pauze vozača zasnovan na proteklom vremenu vožnje. Sistem se sastoji od Go backend servisa, PostgreSQL/PostGIS baze podataka, RabbitMQ posrednika poruka, Valhalla routing engine-a i Flutter mobilne aplikacije, a svi servisi su kontejnerizovani i pokreću se preko docker -compose.

1.1 Motivacija

Motivacija za rad proizlazi iz konkretnog, ponovljivog problema: standardna navigacija ne razlikuje teretno vozilo od putničkog automobila, dok specijalizovana komercijalna rešenja skrivaju logiku odabira rute od korisnika i po pravilu zahtevaju plaćenu licencu i vlasnički skup podataka o putnoj mreži. Cilj ovog rada je da pokaže da se prihvatljivo tačan i **objašnjiv** sistem za rutiranje teretnih vozila može izgraditi nad potpuno otvorenim podacima (OSM) i otvorenim routing engine-om (Valhalla), uz sopstveni algoritamski doprinos koji tim alatima nedostaje: procenu rizika rute specifičnu za teretni saobraćaj i automatsko objašnjenje odstupanja rute od očekivanog, najkraćeg puta.

1.2 Ciljevi i obim rada

Ciljevi rada su:

- proučiti postojeća rešenja za rutiranje teretnih vozila i teorijske osnove distribuiranih sistema i algoritama pretrage najkraćeg puta u grafu;
- pripremiti geografske podatke za teritoriju Republike Srbije iz OSM ekstrakta, sa oznakama relevantnim za teretni saobraćaj;
- implementirati backend servis koji generiše, rangira i objašnjava rutu za dato vozilo, koristeći Valhallu kao routing engine;
- implementirati, kao odvojenu algoritamsku celinu, sopstvenu Dijkstra/A* pretragu nad ograničenim podgrafom, radi evaluacije principa rutiranja bez zavisnosti od Valhalla-e;
- implementirati distribuiranu komunikaciju sistema (REST, asinhrona obrada, WebSocket) i mobilnu aplikaciju za vozače i dispečere;
- testirati implementirano rešenje i evaluirati doprinos sopstvene funkcije rizika u odnosu na "sirov" izbor rute.

Rad se svesno ne bavi punom implementacijom evropske AETR regulative o radnom vremenu i pauzama vozača, elevacionim (nagibnim) rizikom rute, offline režimom rada mobilne aplikacije, podrškom za više država niti bezbednosnim hardeningom na produkcionom nivou — ta ograničenja su namerna i obrazložena u poglavlju 10.

1.3 Struktura rada

Poglavlje 2 daje pregled postojećih komercijalnih i otvorenih rešenja za rutiranje teretnih vozila i teorijske osnove (distribuirani sistemi, algoritmi pretrage najkraćeg puta, OSM podatkovni model).

Poglavlje 3 opisuje korišćene tehnologije.

Poglavlje 4 opisuje arhitekturu sistema.

Poglavlje 5 opisuje pripremu geografskih podataka.

Poglavlje 6 detaljno opisuje algoritamski deo rada — Valhalla truck costing, sopstvenu funkciju rizika, mehanizam objašnjenja rute i sopstvenu Dijkstra/A* implementaciju.

Poglavlje 7 opisuje backend servis i model podataka.

Poglavlje 8 opisuje mobilnu aplikaciju.

Poglavlje 9 opisuje testiranje i evaluaciju.

Poglavlje 10 sadrži zaključak i pravce budućeg rada.

2 Pregled postojećih rešenja i teorijske osnove

2.1 Komercijalna i otvorena rešenja za rutiranje teretnih vozila

Rutiranje teretnih vozila je specijalizovana varijanta opšteg problema pronalaženja puta u grafu, sa dodatnim skupom ograničenja koja zavise od fizičkih karakteristika vozila i tereta. Postojeća rešenja mogu se podeliti u tri grupe.

Komercijalna rešenja sa vlasničkim podacima. PTV navigator i PTV xServer (nemački proizvođač PTV Group) su industrijski standard u logistici teretnog saobraćaja u Evropi; koriste vlasnički skup podataka o putnoj mreži sa ručno unetim ograničenjima za kamione (visina mostova, zabrane za HGV — *Heavy Goods Vehicle* — u naseljima i sl.) i naplaćuju se po licenci. TomTom Truck i pojedini moduli Google Maps Platform-a (Routes API sa parametrima vozila) rade na sličnom principu — visok nivo tačnosti podataka, ali zatvoren izvor, zatvorena cena rute i bez mogućnosti da se razumeju ili prilagode kriterijumi po kojima je ruta izabrana.

Otvoreni routing engine-i. OSRM (*Open Source Routing Machine*), GraphHopper i Valhalla su tri najpoznatija otvorena routing engine-a koji rade nad OpenStreetMap podacima. OSRM je najbrži za rutiranje putničkih vozila (koristi *contraction hierarchies*), ali njegova podrška za teretna vozila je ograničena i uglavnom se svodi na jednostavne profile bez finog podešavanja po osovinskom opterećenju. GraphHopper ima ugrađen truck profil sličan Valhalla-inom i dobru Java/JVM integraciju, ali slabiju podršku za alternativne rute i manje detaljan mehanizam maneuver-a u odgovoru. Valhalla, koji je razvio Mapzen (danas ga održava zajednica okupljena oko Valhalla projekta), izabran je za ovaj rad zbog tri razloga: (1) ima *per-request* truck costing model koji prihvata visinu, širinu, dužinu, masu, osovinsko opterećenje i prevoz opasnog tereta kao parametre HTTP zahteva, bez potrebe da se engine iznova kompajlira ili graf iznova gradi za svako vozilo; (2) podržava vraćanje više alternativnih ruta (*alternates*) u jednom zahtevu, što je preduslov za sopstveni sloj rangiranja opisan u poglavlju 6; (3) odgovor sadrži dovoljno detalja o maneuver-ima (tip, naziv ulice, početni indeks u geometriji rute) da se nad njim može izgraditi mehanizam objašnjenja rute (poglavlje 6.3) bez potrebe za dodatnim pozivima ka engine-u.

Akademski i istraživački radovi. Problem rutiranja vozila sa ograničenjima (*constrained vehicle routing*) je dobro proučen u okviru šire oblasti *Vehicle Routing Problem* (VRP) [1] i njenih varijanti, gde se ograničenja tipično modeluju kao tvrda isključenja grana grafa (vozilo ne može da koristi granu) ili kao dodatni članovi u funkciji cene puta. Ovaj rad se ne bavi klasičnim VRP problemom (raspoređivanjem više vozila na više tura radi minimizacije ukupnog troška flote), već užim problemom rutiranja jednog vozila između dve tačke uz fizička ograničenja i sopstvenu procenu rizika — najbliži akademski okvir je *constrained shortest*

path problem, koji je u poglavlju 2.3 i 6.4 rešen sopstvenom implementacijom Dijkstra i A* algoritma sa isključivanjem grana koje vozilo ne može fizički da koristi.

Tabela 2.1 sumira poređenje.

Tabela 2.1: Poređenje pristupa rutiranju teretnih vozila

Rešenje	Izvor podataka	Truck costing	Alternative rute	Objašnjenje rute	Cena
PTV navigator / xServer	Vlasnički	Da, detaljan	Da	Ne (crna kutija)	Komercijalna licenca
Google Routes API (vozilo)	Vlasnički	Delimično	Da	Ne	Po pozivu (plaćeno)
OSRM	OSM	Ograničen	Da	Ne	Besplatno (open source)
GraphHopper	OSM	Da	Ograničeno	Ne	Besplatno / komercijalna podrška
Valhalla (bez izmena)	OSM	Da, detaljan	Da	Ne	Besplatno (open source)
Ovaj rad (Valhalla+ sopstveni sloj)	OSM	Da (Valhalla)	Da (Valhalla)	Da (sopstveni modul)	Besplatno

2.2 Distribuirani sistemi — osnovni pojmovi

Sistem opisan u ovom radu je, u smislu klasifikacije koju daju Tanenbaum i van Steen [2], **distribuiran sistem**: skup nezavisnih računarskih procesa (Go backend, PostgreSQL, RabbitMQ, Valhalla, Flutter klijenti) koji korisniku deluju kao jedinstvena celina, iako fizički i procesno rade odvojeno, najčešće u sopstvenim kontejnerima. U nastavku su ukratko definisani pojmovi neophodni za razumevanje arhitekture opisane u poglavlju 4 i 7.

Model klijent-server. Osnovni obrazac komunikacije u sistemu: mobilna aplikacija (klijent) šalje HTTP zahtev Go backend servisu (server), koji obrađuje zahtev i vraća odgovor. Sam backend je, u odnosu na Valhallu i bazu podataka, istovremeno klijent — što je uobičajena situacija u slojevitim sistemima, gde jedan proces može biti server u jednoj interakciji i klijent u drugoj.

Sinhrona i asinhrona komunikacija. REST API poziv je *sinhron*: klijent čeka odgovor servera pre nego što nastavi. Za operacije koje ne moraju biti završene odmah da bi klijent nastavio sa radom (npr. proračun predloga pauze vozača nakon što je tura započeta), koristi se *asinhroni* model preko posrednika poruka (RabbitMQ) — proizvođač (backend) objavljuje poruku o događaju i odmah nastavlja dalje, a potrošač (worker proces) je obrađuje kad je slobodan, nezavisno od toga da li je klijent koji je pokrenuo turu još povezan.

Model izdavač-pretplatnik (publish–subscribe). RabbitMQ u ovom sistemu radi kao *topic exchange*: proizvođači objavljuju poruke sa određenim *routing key*-em (npr. `trip.started`), a potrošači se pretplaćuju na red vezan za taj routing key. Ovaj model referencijalno razdvaja proizvođača od potrošača — backend koji objavljuje `trip.started` ne mora znati ništa o tome ko (ili da li uopšte neko) sluša taj događaj, što olakšava dodavanje novih potrošača bez izmene postojećeg koda (detaljnije u poglavlju 7.4).

Real-time komunikacija (WebSocket). REST model zahteva da klijent inicira svaki zahtev, što ga čini neprikladnim za slanje podataka od servera ka klijentu bez zahteva (npr. pozicija vozila koja se menja svake sekunde). WebSocket protokol (RFC 6455 [3]) rešava taj problem uspostavljanjem trajne, dvosmerne veze preko jednog TCP soketa nakon početnog HTTP *upgrade* zahteva — server u svakom trenutku može poslati poruku klijentu bez čekanja na njegov zahtev. U ovom sistemu se koristi za praćenje pozicije vozila uživo i za dopisivanje (chat) u realnom vremenu (poglavlje 7.5).

Transparentnost i pouzdanost. Od četiri klasična cilja distribuiranog sistema — povezanost, transparentnost, otvorenost i skalabilnost [2] — u ovom radu je najrelevantnija *transparentnost otkaza (fault transparency)*: sistem je dizajniran da otkaz jedne komponente (npr. privremeni gubitak RabbitMQ konekcije od strane worker procesa) ne obori ostatak sistema, već da se poruka ponovo isporuči (*message redelivery*, videti poglavlje 7.4) kada se komponenta ponovo poveže. S obzirom na obim rada (jedan backend, jedna baza, jedan message broker — bez replikacije ijedne komponente), sistem ne postiže skalabilnost po veličini u produkcionom smislu, što je namerno ograničenje obrazloženo u poglavlju 10.

2.3 Grafovi i algoritmi najkraćeg puta

Putna mreža se prirodno modeluje kao usmereni, težinski graf $G = (V, E)$, gde skup čvorova V predstavlja tačke na putu (raskrsnice, prelome geometrije), a skup grana E predstavlja segmente puta između susednih čvorova, sa težinom koja predstavlja cenu prelaska te grane (najčešće rastojanje u metrima, vreme u sekundama ili neka izvedena veličina). Problem rutiranja se svodi na pronalaženje puta P od početnog čvora s do ciljnog čvora t koji minimizuje zbir težina grana na putu, uz eventualna dodatna ograničenja (izuzete grane).

Dijkstra algoritam. Dijkstrin algoritam [4] pronalazi najkraći put od jednog čvora do svih ostalih čvorova u grafu sa nenegativnim težinama grana, koristeći greedy pristup: u svakom koraku bira neposećeni čvor sa najmanjom trenutno poznatom cenom od početka,

”fiksira” tu cenu kao konačnu i ažurira cene njegovih suseda (*relaxation*). Algoritam se efikasno implementira uz prioritetni red (u ovom radu binarna gomila, *container/heap* iz Go standardne biblioteke), sa vremenskom složenošću $O((|V|+|E|) \log |V|)$. Kada je poznat samo jedan ciljani čvor (a ne svi čvorovi grafa), pretraga se može prekinuti čim je ciljani čvor skinut sa reda — ta optimizacija je korišćena u implementaciji opisanoj u poglavlju 6.4.

A* algoritam. A* algoritam [5] je uopštenje Dijkstre koje uvodi *heuristiku* $h(n)$ — procenu preostale cene od čvora n do cilja — i usmerava pretragu prioritetno ka čvorovima za koje je zbir $f(n) = g(n) + h(n)$ (stvarna cena do sada plus procena preostale cene) najmanji. Ako je heuristika *dopustiva* (nikad ne preceni stvarnu preostalu cenu), A* garantovano pronalazi optimalan put, uz manji broj posećenih čvorova od Dijkstre u praksi. U ovom radu je kao heuristika korišćena haversine udaljenost (linija vazdušne udaljenosti po sferi Zemlje) od trenutnog čvora do cilja, podeljena maksimalnom mogućom brzinom kretanja po grani — dopustiva heuristika jer nikad ne potcenjuje stvarni put duž putne mreže, koji je uvek duži ili jednak vazdušnoj liniji.

Ograničena (constrained) pretraga. Fizička ograničenja vozila (visina, masa, prevoz opasnog tereta) se u ovom radu modeluju kao *tvrda isključenja* grane iz pretrage: grana koja ne zadovoljava profil vozila jednostavno se ne razmatra kao prelaz, čime pretraga po definiciji nikad ne predloži fizički neprohodnu rutu. Ovaj princip je identičan principu na kome interno počiva Valhalla-in truck costing model — razlika je u tome da Valhalla ta isključenja primenjuje nad grafom cele putne mreže, obogaćenim brojnim heuristikama (vreme prolaska, tipovi puta, istorijski podaci o saobraćaju), dok sopstvena implementacija u ovom radu (poglavljje 6.4) to radi nad znatno manjim, ograničenim podgrafom, isključivo radi transparentne demonstracije mehanizma.

2.4 OpenStreetMap podatkovni model i oznake za teretni saobraćaj

OpenStreetMap (OSM) je projekat slobodne, uređivačke geografske baze podataka sveta, u kojoj se putna mreža, objekti i granice modeluju preko tri osnovna geometrijska primitiva: **node** (tačka, definisana geografskom širinom i dužinom), **way** (uređena lista node-ova, predstavlja liniju ili poligon — npr. jedan segment puta) i **relation** (grupisanje node-ova, way-eva i drugih relation-a — npr. skup segmenata koji zajedno predstavljaju jednu magistralnu rutu ili ograničenje skretanja). Svaki od ova tri primitiva može imati proizvoljan skup **tag**-ova — parova ključ-vrednost koji opisuju njegova svojstva.

Za teretni saobraćaj relevantni su, između ostalih, sledeći tagovi (dokumentovani na OSM Wiki-ju [6]):

- **maxheight**, **maxweight**, **maxwidth** — na way-u, ograničenje visine/mase/širine za tu deonicu (npr. nadvožnjak, tunel);
- **hgv** — da li je way dozvoljen za teška teretna vozila (*Heavy Goods Vehicle*), sa vred-

nostima poput `yes`, `no`, `destination`;

- `hazmat` — ograničenje za vozila koja prevoze opasan teret;
- `maxaxleload` — ograničenje osovinskog opterećenja, relevantno za mostove;
- `surface` — vrsta podloge (asfalt, kolovoz, tucanik i sl.), indirektno relevantno za rizik (loša podloga + veliko opterećenje = veći rizik);
- `barrier` na node-u (npr. `barrier=height_restrictor`, `barrier=lift_gate`) — tačkasta prepreka koja može imati sopstveni `maxheight` tag, različit od tag-a way-a na kome se nalazi (npr. rampa niske visine na inače visokom putu);
- `amenity=fuel`, `amenity=parking`, `highway=rest_area` — node-ovi koji predstavljaju benzinske stanice, parkinge i odmarališta, relevantni za modul predloga pauze vozača (poglavlje 7.4).

Bitna karakteristika OSM podataka, relevantna za poglavlje 5, je da su ograničenja poput `maxheight` i `barrier` gotovo uvek zavedena na **way** nivou (za deonicu puta), dok su tačkaste prepreke (rampe, stubovi) zavedene na **node** nivou — što znači da alat koji filtrira OSM ekstrakt mora eksplicitno čuvati oba tipa tagova (way i node), jer se u suprotnom gubi tačno ona informacija koja je najrelevantnija za bezbednost teretnog vozila.

3 Korišćene tehnologije

3.1 Go

Backend servis je implementiran u programskom jeziku Go (Golang) [7], koji je razvio Google 2009. godine. Go je statički tipiziran, kompajliran jezik sa automatskim upravljanjem memorijom (*garbage collection*) i ugrađenom podrškom za konkurentno programiranje preko *goroutine*-a (lakih, korisnički upravljanih niti) i *channel*-a (tipiziranih kanala za komunikaciju između goroutine-a). Za ovaj rad je Go izabran iz tri razloga: (1) standardna biblioteka već sadrži kompletan HTTP server (`net/http`, uz Go 1.22+ rutiranje po putanji i HTTP metodu preko `http.ServeMux`, korišćeno u `httpapi` paketu), bez potrebe za eksternim veb okvirom; (2) goroutine model konkurentnosti je pogodan za istovremeno opsluživanje mnogo WebSocket konekcija (poglavlje 7.5) i pozadinskog RabbitMQ potrošača (poglavlje 7.4) unutar istog procesa, bez složene infrastrukture za konkurentnost kakva bi bila potrebna u jednonitskim okruženjima; (3) statička tipizacija i kompajliranje u jedan izvršni fajl pogoduju jednostavnom kontejnerizovanju (poglavlje 3.7).

3.2 Flutter i Dart

Mobilna aplikacija je implementirana u Flutter okviru [8], koji koristi programski jezik Dart (oba razvijena od strane Google-a). Flutter kompajlira aplikaciju u nativni kod za Android i iOS iz jedinstvene baze izvornog koda, a korisnički interfejs se ne oslanja na nativne widgete platforme već na sopstveni *rendering engine* (Skia/Impeller), čime se postiže vizuelna konzistentnost između platformi. Za potrebe ovog rada korišćene su, između ostalih, biblioteke `geolocator` (pristup GPS senzoru uređaja), `flutter_map` odnosno OSM/vector tile prikaz mape, `web_socket_channel` za WebSocket konekcije i `google_sign_in` za prijavu preko Google naloga.

3.3 Valhalla

Valhalla [9] je otvoreni (open source) routing engine specijalizovan za rad nad OpenStreetMap podacima, originalno razvijen u kompaniji Mapzen. Za razliku od routing engine-a koji graf i cenu puta fiksiraju u trenutku izgradnje (tzv. *preprocessing*, kakav koristi npr. OSRM sa *contraction hierarchies*), Valhalla dozvoljava da se većina parametara cene puta (costing) prosledi **po zahtevu** (*per-request*), uključujući ceo truck profil vozila — što znači da nije potrebno iznova graditi graf niti pokretati novu instancu servisa za svako vozilo ili promenu profila. Valhalla graf se organizuje u hijerarhijske "pločice" (*tiles*), izgrađe-

ne alatom `valhalla_build_tiles` iz `.osm.pbf` ekstrakta (poglavlje 5.2), a HTTP API (`valhalla_service`) prihvata zahteve na endpoint-u `/route` sa JSON telom koje uključuje lokacije, tip costing modela (`truck`) i njegove opcije. Detaljna upotreba Valhalla-inog truck costing modela u ovom radu opisana je u poglavlju 6.1.

3.4 PostgreSQL i PostGIS

PostgreSQL je korišćen kao relacionalna baza podataka za skladištenje vozila, vozača, tura i pratećih entiteta (poglavlje 7.2). PostGIS [10] je ekstenzija za PostgreSQL koja dodaje geografske tipove podataka i prostorne upite (npr. "sve tačke u krugu od 3km oko date koordinate"). Iako trenutna implementacija čuva geometriju rute kao enkodovan tekstualni *polyline* (a ne PostGIS `geometry` kolonu — videti poglavlje 7.2 i ograničenja u poglavlju 10), PostGIS ekstenzija je uključena u korišćenu Docker sliku (`postgis/postgis:16-3.4-alpine`) kao osnova za prostorne upite koje bi budući razvoj (npr. pretraga "najbliže odmaralište" direktno u bazi umesto u memoriji, poglavlje 7.4) mogao iskoristiti bez promene infrastrukture.

3.5 RabbitMQ

RabbitMQ [11] je posrednik poruka (*message broker*) koji implementira AMQP 0-9-1 protokol. U ovom sistemu se koristi *topic exchange* (razmenjivač po temi) nazvan `trip.events`: proizvođač (backend) objavljuje poruku sa *routing key*-em `trip.started` kada vozač pokrene turu, a worker proces (poglavlje 7.4) je preuzima iz reda vezanog za taj routing key, obrađuje je i upisuje rezultat (predlog pauze) u bazu. RabbitMQ je izabran zbog ugrađene podrške za potvrdu obrade poruke (*acknowledgment*) i automatsko ponovno slanje neuspešno obrađene poruke (*requeue*), što sistemu daje otpornost na privremeni pad worker procesa bez gubitka podataka o događaju.

3.6 WebSocket

Za real-time komunikaciju (praćenje pozicije vozila uživo i chat) korišćena je biblioteka `gorilla/websocket` na backend strani i `web_socket_channel` na Flutter strani, obe implementacije WebSocket protokola (RFC 6455 [3]) nad HTTP/1.1 *upgrade* mehanizmom. Detalji implementacije dati su u poglavlju 7.5.

3.7 Docker i docker-compose

Svi serverski delovi sistema (Go backend, PostgreSQL, RabbitMQ, Valhalla) su kontejnerizovani Docker slikama [12] i orkestrisani preko `docker-compose`, alata koji na osnovu deklarativnog YAML fajla pokreće, povezuje u zajedničku mrežu i redosledom health-check

zavisnosti startuje više kontejnera jednom komandom (`docker compose up`). Ovo omogućava da se ceo backend deo sistema pokrene na bilo kojoj mašini sa Docker-om instaliranim, bez ručne instalacije PostgreSQL-a, RabbitMQ-a ili Valhalla-e — što je posebno važno za predvidljivost demonstracije rada (poglavljje 9). Detalji konfiguracije dati su u poglavljju 4.2.

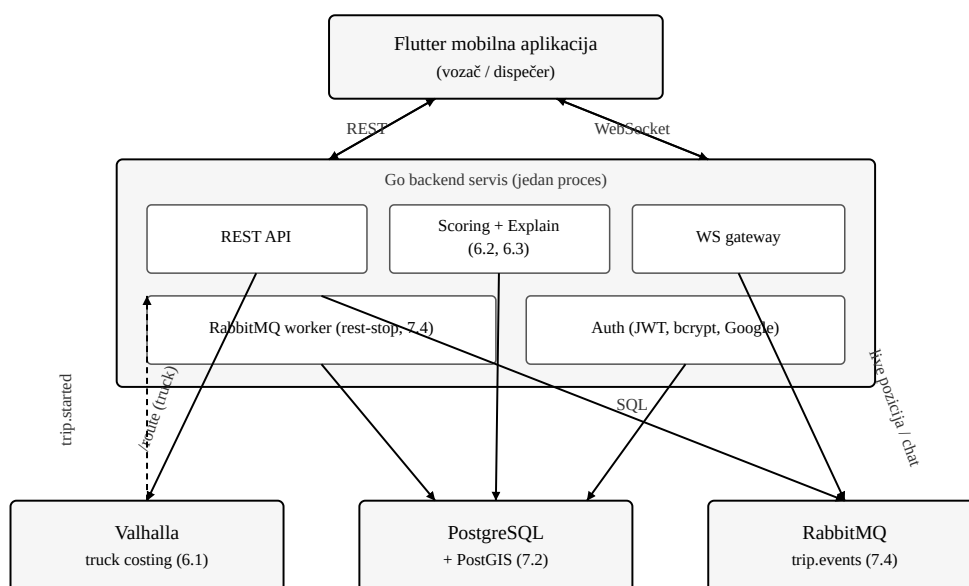
3.8 Ostale biblioteke i servisi

- **JWT (JSON Web Token, RFC 7519 [13])** — format za autentifikacione tokene korišćen preko biblioteke `golang-jwt/jwt`, sa HS256 potpisom; nosi identitet vozača i broj verzije tokena (`token_version`), koji se koristi za "odjavu sa svih uređaja" bez potrebe za posebnom listom opozvanih tokena (poglavljje 4.3).
- **bcrypt** — algoritam za heširanje lozinki sa ugrađenom "solju" i podesivim faktorom rada, korišćen umesto čuvanja lozinki u čistom tekstu ili prostog heša.
- **Google Sign-In / JWKS** — prijava preko Google naloga verifikuje se ručnom proverom Google-ovog ID token-a nad javnim skupom ključeva (JWKS) preko biblioteke `MicahParks/keyfunc`, bez uvođenja celog Firebase SDK-a kao zavisnosti.
- **Nominatim [14]** — javni servis za geokodiranje (pretvaranje adrese u koordinate i obrnuto) projekta `OpenStreetMap`, korišćen za pretragu adresa u mobilnoj aplikaciji (poglavljje 7.1), sa ograničenjem brzine poziva (`throttling`) u skladu sa politikom korišćenja servisa.
- **goose [15]** — alat za upravljanje migracijama šeme baze podataka preko numerisanih SQL fajlova (poglavljje 7.2), koji garantuje da se svaka migracija primeni tačno jednom.
- **osmium-tool** — komandolinijski alat za filtriranje i obradu `.osm/.osm.pbf` fajlova, korišćen u pripremi geografskih podataka (poglavljje 5.1).

4 Arhitektura sistema

4.1 Pregled arhitekture i tok podataka

Sistem se sastoji od pet nezavisnih procesa: Flutter mobilne aplikacije (klijent, vozač ili dispečer), Go backend servisa (centralna tačka logike), Valhalla routing engine-a, PostgreSQL/PostGIS baze podataka i RabbitMQ posrednika poruka. Slika 4.1 prikazuje njihov međusobni odnos i osnovni tok podataka pri planiranju i praćenju jedne ture.



Isprekidana linija: asinhrona komunikacija (RabbitMQ). Pune linije: sinhroni HTTP/SQL/WebSocket pozivi.

Slika 4.1: Arhitektura sistema — komponente i tok podataka

Tok pri planiranju rute je sledeći: mobilna aplikacija šalje `POST /api/v1/routes` sa profilom vozila, polazištem i odredištem; backend mapira profil vozila u Valhalla-in truck costing model (poglavlje 6.1) i traži do tri alternativne rute; svaka alternativa se rangira sopstvenom funkcijom rizika (poglavlje 6.2), a za izabranu (najbolje rangiranu) rutu se, ako odstupa od geometrijski najkraćeg puta, generiše tekstualno objašnjenje (poglavlje 6.3); odgovor sa geometrijom, rizikom i objašnjenjem se vraća klijentu. Kada vozač zaista pokrene turu (`POST /api/v1/trips`, zatim prelazak u status `in_progress`), backend objavljuje `trip.started` događaj na RabbitMQ, koji worker proces preuzima i računa predlog pauze (poglavlje 7.4); istovremeno se otvara WebSocket veza (`GET /ws/trips/{id}`) preko koje se, dok vozač šalje periodične GPS pozicije (`POST /api/v1/trips/{id}/position`), te pozicije prosleđuju svim zainteresovanim klijentima uživo — vozačevom sopstvenom ekranu i, ako postoji, njegovom dispečeru koji sistem preko sopstvene, paralelne WebSocket

konekcije prati na jednoj mapi (poglavlje 7.5, 8.2).

Sistem sledi troslojnu podelu odgovornosti sličnu klasičnoj veb arhitekturi [2]: sloj prezentacije (Flutter aplikacija), sloj obrade (Go backend, uključujući scoring i explain module) i sloj perzistencije (PostgreSQL). Razlika u odnosu na uobičajenu troslojnu veb aplikaciju je što backend za svaki zahtev za rutu sam postaje klijent eksternog servisa (Valhalla), a deo obrade (predlog pauze) je pomeren u nezavisan, asinhron proces (worker), radi rasterećenja glavne HTTP niti obrade zahteva.

4.2 Kontejnerizacija i orkestracija servisa

Svi serverski delovi sistema definisani su u jednom `docker-compose.yml` fajlu i pokreću se komandom `docker compose up`. Listing 4.1 prikazuje definiciju PostgreSQL servisa, sa health-check-om koji ostali servisi (backend) koriste kao uslov za sopstveni start preko `depends_on: condition: service_healthy` — čime se garantuje da backend ne pokuša konekciju na bazu koja još nije spremna za prihvatanje konekcija.

```
postgres:
  image: postgis/postgis:16-3.4-alpine
  environment:
    POSTGRES_USER: hvr
    POSTGRES_PASSWORD: hvr_dev_password
    POSTGRES_DB: hvr
  ports:
    - "5433:5432"
  volumes:
    - pgdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U hvr -d hvr"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Listing 4.1: Definicija PostgreSQL/PostGIS servisa u `docker-compose.yml`

Tabela 4.1 sumira sve servise definisane u `docker-compose.yml`, njihovu ulogu i portove koje izlažu na host mašini.

Tabela 4.1: Servisi definisani u `docker-compose.yml`

Servis	Slika	Uloga	Port (host)
postgres	postgis/postgis:16	Relacijska baza + PostGIS ekstenzija	5433 → 5432
rabbitmq	rabbitmq:3-management	Potrebne poruka (AMQP) + web konzola	5672, 15672
valhalla	ghcr.io/valhalla/valhalla	HTTP routing servis (valhalla_service)	8002
valhalla-build	ghcr.io/valhalla/valhalla-build	Uzorkovanje i izgradnja grafa iz OSM ekstrakta (profil build, ne pokreće se pri običnom up)	—
backend	sopstveni Go build	REST API, WebSoc- ket gateway, worker	8080

`valhalla-build` je definisan sa Docker Compose *profilom* build, što znači da se ne pokreće automatski pri `docker compose up` — pokreće se izričito (`docker compose --profile build up valhalla-build`) samo kada treba ponovo izgraditi graf (npr. nakon promene OSM ekstrakta, poglavlje 5), i radi sa `network_mode: none` jer mu za samu izgradnju grafa nije potrebna mrežna konekcija, čime se smanjuje površina za eventualni bezbednosni problem tog koraka. Mobilna aplikacija i sopstveni algoritamski modul (poglavlje 6.4) nisu deo `docker-compose.yml` — Flutter aplikacija se pokreće lokalno na razvojnoj mašini ili emulatoru, a algoritamski modul je čista Go biblioteka koja se pokreće isključivo kroz testove, bez sopstvenog servisa (obrazloženo u poglavlju 6.4).

4.3 Sigurnost i autentifikacija

Autentifikacija vozača i dispečera zasniva se na JWT tokenima (RFC 7519 [13]) potpisanim HMAC SHA-256 algoritmom (HS256). Nakon prijave (`POST /auth/login` sa korisničkim imenom i lozinkom, ili `POST /auth/google` sa Google ID tokenom), backend izdaje token koji, pored identifikatora vozača, nosi i celobrojno polje `token_version`. Ovo polje se upoređuje sa vrednošću `token_version` iz baze na svakom autentifikovanom zahtevu (`RequireAuth` middleware) — kada vozač zatraži odjavu sa svih uređaja (`POST /auth/logout-all`), backend samo inkrementira `token_version` u bazi, čime svi ranije izdati tokeni (koji nose stariju vrednost) trenutno postaju nevažeći, bez potrebe za posebnom,

rastućom tabelom opozvanih tokena (*blocklist*) koja bi morala da se proverava na svakom zahtevu.

Lozinke se ne čuvaju u čistom tekstu, već kao bcrypt heš, algoritam koji ugrađuje nasumičnu "sol" po lozinci (čime dve identične lozinke dva različita korisnika dobijaju različit heš) i podesivi faktor rada (koji direktno određuje koliko dugo traje jedan pokušaj heširanja, pa time i koliko dugo traje jedan pokušaj brute-force napada). Nalozi kreirani preko Google prijave nemaju lozinku (`password_hash` je `NULL`) — identifikuju se preko stabilnog Google identifikatora naloga (`google_sub`, `sub` polje iz Google ID tokena), koji backend verifikuje samostalno preko Google-ovog javnog skupa ključeva (JWKS), bez uvođenja Firebase SDK-a kao zavisnosti.

Za obnovu zaboravljene lozinke (`POST /auth/forgot-password`) i verifikaciju email adrese (`GET /auth/verify-email`) koristi se model jednokratnog, vremenski ograničenog tokena zapisanog u posebnoj tabeli (`password_reset_tokens`, `email_verification_tokens`) sa poljima `expires_at` i `used_at` — link poslat emailom prestaje da važi i po isteku vremena i po jednokratnoj upotrebi, a backend razlikuje ta dva slučaja radi jasnije poruke korisniku. Slanje emaila (link za verifikaciju/reset) realizovano je preko `net/smtp` paketa iz Go standardne biblioteke, bez eksternog transakcionog email API-ja — servis je `no-op` (ne šalje ništa, samo zapisuje u log) ako promenljiva okruženja `SMTP_HOST` nije podešena, čime ostatak sistema radi i bez konfigurisanog email servisa (npr. u razvojnom okruženju).

Sistem u trenutnom obliku ima namerno ograničen bezbednosni obim, obrazložen u poglavlju 10 — `WebSocket CheckOrigin` proveru je trenutno propustljiva (prihvata sve `origin`e), pošto je aplikacija zamišljena kao rad sa jednim poznatim, poverljivim klijentom (sopstvena Flutter aplikacija), a ne kao javni, višezakupčani (`multi-tenant`) servis.

5 Priprema geografskih podataka — OSM ekstrakcija za teretna vozila u Srbiji

5.1 Izvor podataka i osmium filter

Osnovni izvor geografskih podataka je Geofabrik-ov dnevno ažuriran ekstrakt OpenStreetMap podataka za Srbiju (`serbia-latest.osm.pbf`) [16], preuzet automatizovanim `update-osm.sh` skriptom koja, pored preuzimanja, proverava integritet fajla preko MD5 kontrolne sume koju Geofabrik objavljuje uz sam ekstrakt. Kompletan `.osm.pbf` fajl za Srbiju sadrži ogroman broj tagova nepotrebnih za rutiranje teretnih vozila (npr. sve poljoprivredne parcele, granice popisnih krugova, turističke oznake), pa se filtrira alatom `osmium-tools` [17] pre nego što se prosledi Valhalla-i.

Listing 5.1 prikazuje tačnu `osmium tags-filter` komandu korišćenu u projektu.

```
osmium tags-filter \  
  serbia-latest.osm.pbf \  
  w/highway w/maxheight w/maxweight w/maxwidth \  
  w/hgv w/hazmat w/bridge w/tunnel \  
  w/surface w/maxspeed \  
  n/amenity=fuel,parking n/highway=rest_area n/barrier \  
  r/type=restriction \  
  --output serbia-hvt.osm.pbf --overwrite
```

Listing 5.1: Osmium filter za ekstrakciju podataka relevantnih za teretni saobraćaj

Filter čuva way-eve (`w/`) sa tagovima relevantnim za geometriju i ograničenja puta (`highway`, `maxheight`, `maxweight`, `maxwidth`, `hgv`, `hazmat`, `bridge`, `tunnel`, `surface`, `maxspeed`), relacije (`r/`) tipa `restriction` (ograničenja skretanja) i, što je bilo kritično ispraviti tokom razvoja (videti dalje u ovom poglavlju), node-ove (`n/`) sa tagovima `amenity=fuel`, `amenity=parking`, `highway=rest_area` i `barrier`. Way i node su, kako je objašnjeno u poglavlju 2.4, dva različita nivoa na kojima OSM podaci nose ograničenja relevantna za teretna vozila — way-evi obično nose ograničenje za celu deonicu (npr. `maxheight` na mostu), a node-ovi tačkastu prepreku (npr. rampu ili stub na ulazu u parking).

Napomena o razvoju filtera. Prva verzija filtera korišćena u ranoj fazi razvoja sadržala je samo `w/` i `r/` pravila, bez ijednog `n/` pravila — što je značilo da su svi node-ovi sa `amenity=fuel`, `amenity=parking`, `highway=rest_area` i `barrier` tagovima bili odbaceni pri filtriranju, iako su neophodni za modul predloga pauze vozača (poglavlje 7.4). Ovaj propust je otkriven i ispravljen tokom pripreme podataka za rad — dodavanjem tri `n/` pravila

filtrirani fajl je narastao sa 68 121 223 na 68 402 528 bajtova, a Valhalla graf je ponovo izgrađen sa ispravljenim skupom podataka. Ovaj slučaj ilustruje opštu lekciju vezanu za pripremu OSM podataka: filter koji izgleda kompletan gledano samo kroz way tagove tiho odbacuje čitavu kategoriju informacija zavedenih na node nivou, a greška se ne manifestuje kao pad sistema već kao *nedostajuća* funkcionalnost koja se lako previdi dok se posebno ne testira.

5.2 Izgradnja Valhalla grafa

Filtriran `.osm.pbf` fajl se prosleđuje alatu `valhalla_build_tiles`, koji ga parsira i izgrađuje hijerarhijski graf organizovan u pločice (*tiles*) — trajni format koji Valhalla HTTP servis (`valhalla_service`) učitava pri startu i koristi za sve naredne `/route` zahteve. Ovaj korak je u sistemu izdvojen u poseban Docker Compose servis (`valhalla-build`, poglavlje 4.2), koji se pokreće samo kada treba ponovo izgraditi graf, a ne pri svakom pokretanju sistema (jer izgradnja traje nekoliko minuta i, s obzirom na to da se OSM podaci ne menjaju u toku jedne demonstracije rada, nije potrebno da se ona izvršava pri svakom `docker compose up`).

Nakon ispravke filtera opisane u 5.1, ponovo izgrađen graf za teritoriju Republike Srbije sadrži 898 337 čvorova i 2 216 948 usmerenih ivica (grana). Ovaj graf koristi isključivo Valhalla, preko HTTP `/route` poziva (poglavljje 6.1) — nije direktno dostupan Go backend-u niti bilo kom drugom delu sistema, čime se čitava logika parsiranja OSM podataka i izgradnje efikasne rutne strukture prepušta Valhalla-i, u skladu sa odlukom (obrazloženom u poglavlju 6) da se produkcionni put ne oslanja na sopstvenu implementaciju pretrage nad grafom cele države.

5.3 Ekstrakcija podataka o odmaralištima

Za modul predloga pauze vozača (poglavljje 7.4) backend ne poziva Valhallu, već direktno učitava, pri pokretanju, sve node-ove sa tagovima `amenity=fuel`, `amenity=parking` i `highway=rest_area` iz filtriranog OSM ekstrakta (istog fajla korišćenog za izgradnju Valhalla grafa, poglavljje 5.1) u memoriju procesa — oko 2000 node-ova za teritoriju Srbije. Ova lista se pri svakom predlogu pauze pretražuje geometrijski: kandidat mora biti unutar zadatog radijusa od tačke na ruti u kojoj bi vozilo teorijski bilo nakon isteka praga vožnje (poglavljje 7.4), i mora zaista biti u koridoru trase (a ne, na primer, geometrijski najbliža tačka vazdušnom linijom koja se nalazi na potpuno drugom putu). Rezultat pretrage dodatno favorizuje omiljene ili brend-specifične lokacije vozača, a za vozila koja prevoze opasan teret preferira benzinske stanice nad parkinzima u granicama tolerancije rastojanja — detalji ove logike dati su u poglavlju 7.4.

6 Algoritam rutiranja prilagođen vozilu

Ovo poglavlje je centralni algoritamski doprinos rada. Sistem koristi **dvoslojni pristup**: (1) u produkcionom putu, Valhalla generiše fizički dopustive alternativne rute preko svog truck costing modela (6.1), nad kojima sopstveni Go modul dodaje rangiranje po riziku (6.2) i objašnjenje odstupanja rute (6.3), pošto to Valhalla nativno ne radi; (2) kao odvojena, algoritamska celina namenjena evaluaciji i demonstraciji principa, implementirana je sopstvena Dijkstra/A* pretraga nad ograničenim podgrafom putne mreže, direktno nad OSM podacima, bez ikakve zavisnosti od Valhalla-e (6.4). Ovaj pristup je posledica realne procene obima rada: reimplementacija efikasnog nacionalnog routing engine-a nad grafom od skoro milion čvorova (poglavljje 5.2) nije realan cilj jednog diplomskog rada, dok je razumevanje i demonstracija principa na kojima takav engine počiva — kroz sopstvenu, testiranu implementaciju nad manjim, ali realnim podgrafom — sasvim ostvarivo i akademski relevantno.

6.1 Valhalla truck costing

Valhalla-in truck costing model prihvata profil vozila kao deo tela HTTP /route zahteva, bez potrebe da se graf iznova izgradi za svako vozilo. Listing 6.1 prikazuje tipičan zahtev korišćen u ovom sistemu.

```
POST /route
{
  "locations": [
    {"lat": 44.8, "lon": 20.4},
    {"lat": 45.25, "lon": 19.85}
  ],
  "costing": "truck",
  "costing_options": {
    "truck": {
      "height": 4.0,
      "width": 2.55,
      "length": 16.5,
      "weight": 40,
      "axle_load": 11.5,
      "hazmat": false
    }
  },
  "alternates": 2
}
```

Listing 6.1: Zahtev za truck-costed rutu sa alternativama (visina/širina/dužina u metrima, masa/osovinsko opterećenje u tonama)

Backend mapira sopstveni `TruckProfile` (u SI jedinicama — metrima i kilogramima, radi konzistentnosti sa ostatkom sistema) u Valhalla-ine jedinice (metri za dimenzije, metrički toni za masu), i parsira odgovor u sopstvenu strukturu `RouteCandidate` koja, pored rastojanja i trajanja, izdvaja i broj manevara, udeo puta na magistralnim/autoputskim deonicama (`HighwayRatio`), prisustvo trajekta ili putarine, kao i nazive ulica i tačke svakog manevara — signale koje ne izlaže sam sažetak rute, ali su neophodni za sloj rangiranja (6.2) i objašnjenja (6.3). Listing 6.2 prikazuje deo implementacije koja šalje zahtev i mapira profil vozila.

```
// TruckProfile mirrors Valhalla's truck costing_options, in SI units (
    meters, kilograms).
type TruckProfile struct {
    HeightM float64
    WidthM float64
    LengthM float64
    WeightKg float64
    AxleLoadKg float64
    Hazmat bool
}

func (c *Client) RouteAlternates(ctx context.Context, origin, destination
    LatLon,
    profile TruckProfile, numAlternates int) ([]RouteCandidate, error) {
    body := routeRequest{
        Locations: []LatLon{origin, destination},
        Costing: "truck",
        CostingOptions: map[string]truckCostingOpt{
            "truck": {
                Height: profile.HeightM,
                Width: profile.WidthM,
                Length: profile.LengthM,
                Weight: profile.WeightKg / 1000, // Valhalla expects metric tons
                AxleLoad: profile.AxleLoadKg / 1000,
                Hazmat: profile.Hazmat,
            },
        },
        Alternates: numAlternates,
    }
}
```

```
// ... serijalizacija, HTTP POST na baseURL+"/route", parsiranje odgovora
}
```

Listing 6.2: `valhalla.Client.RouteAlternates` — mapiranje profila vozila u Valhalla `costing_options` (`backend/internal/valhalla/client.go`)

Ograničenje ovog pristupa je da odgovor običnog `/route` poziva ne izlaže *koja tačno* ivica grafa je isključena niti njenu tačnu vrednost ograničenja (npr. tačnu visinu nadvožnjaka) — ta informacija bi zahtevala poziv Valhalla-inog `/trace_attributes` endpoint-a nad već izračunatom rutom, ili direktan pristup OSM podacima nezavisno od Valhalla-e. Ovo ograničenje je direktno motivisalo modul opisan u 6.4, koji, radeći nad sopstvenim, manjim grafom, ima pristup tačnim OSM tagovima svake ivice.

6.2 Prilagođena funkcija rizika (scoring)

Valhalla po zahtevu vraća do tri alternativne rute, ali ih ne rangira po bilo kom kriterijumu specifičnom za teretni saobraćaj — sve tri su, sa njene tačke gledišta, fizički dopustive. Paket `scoring` dodaje taj nedostajući sloj: svaku alternativu bodira heurističkom funkcijom rizika koja kombinuje rastojanje, broj manevara, udeo puta van magistralnih deonica, prisustvo trajekta/putarine, procenu potrošnje goriva u odnosu na masu vozila, broj "oštrih" manevara (proxy za rizik pomeranja tereta) i blizinu omiljenih/brend benzinskih stanica vozača — a zatim vraća alternative sortirane od najboljeg (najmanji rizik) ka najgorem rezultatu.

Svaka dimenzija skalira se prema preferenci vozača, izraženoj kao ceo broj od 1 do 5 (gde je 3 neutralna, podrazumevana vrednost) preko faktora `priority/3` — vozač koji nikad ne podesi svoje preference dobija rezultat identičan fiksnoj, uravnoteženoj formuli. Listing 6.3 prikazuje centralnu funkciju bodovanja.

```
func score(c valhalla.RouteCandidate, prefs Preferences, vehicleWeightKg,
    fastestDurationMin float64, preferredStops []valhalla.LatLon) float64 {
    var timeTerm float64
    if fastestDurationMin > 0 {
        timeTerm = (c.DurationMin - fastestDurationMin) / fastestDurationMin *
            timeScale
    }
    highwayTerm := (1 - c.HighwayRatio) * nonHighwayScale
    fuelTerm := fuelProxy(c, vehicleWeightKg)
    cargoTerm := float64(c.SharpManeuverCount) * sharpManeuverWt

    s := scalar(prefs.TimePriority)*timeTerm +
        scalar(prefs.HighwayPriority)*highwayTerm +
        scalar(prefs.FuelPriority)*fuelTerm +
        scalar(prefs.CargoPriority)*cargoTerm +
```

```

float64(c.ManeuverCount)*maneuverWeight

if c.HasFerry {
    s += ferryPenalty
}
if c.HasToll {
    s += tollPenalty
}
s += preferredStopDiscount(c.Shape, preferredStops)
return s
}

```

Listing 6.3: Funkcija bodovanja rute (backend/internal/scoring/scoring.go) — manji rezultat je bolji

Vrednost `timeTerm` je relativno kašnjenje kandidata u odnosu na najbrži kandidat u istom skupu alternativa (0% ako je kandidat najbrži), a `fuelProxy` je relativna procena ”potrošnje” izvedena iz rastojanja i mase vozila (nema pristupa podacima o nagibu puta, pa ovo nije stvarna potrošnja u litrima, već relativan signal za poređenje kandidata međusobno). Autor koda je u komentarima izričito naglasio da su bazne težine (`maneuverWeight`, `nonHighwayScale`, `ferryPenalty` itd.) prva heuristička procena, nekalibrisana prema stvarnim podacima o potrošnji ili nesrećama — što je pošteno i namerno ograničenje, razmatrano dalje u poglavlju 9.3 kroz konkretan primer greške koju je ova formula ispravila.

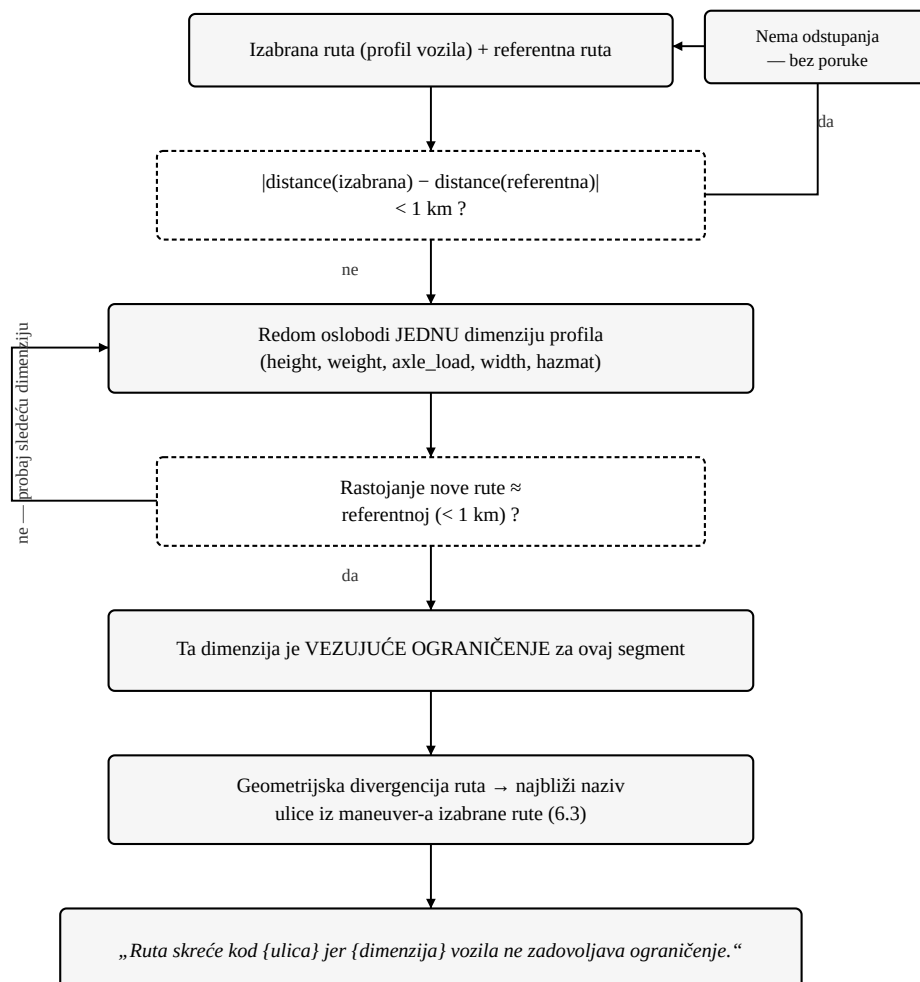
Motivacija dodavanja vremenskog člana. Rana verzija formule nije sadržala `timeTerm` — bodovala je isključivo na osnovu udela magistralne deonice (`highwayTerm`) i broja manevara. Testiranjem na konkretnoj ruti (Radalj, poglavlje 9.3) uočeno je da takva formula bira rutu koja je 47% duža i 30% sporija od alternative samo zato što ima nešto povoljniji odnos magistralne i lokalne deonice puta — očigledno pogrešan ishod sa stanovišta stvarnog transportnog troška. Dodavanje `timeTerm`-a, koji direktno kažnjava kandidata srazmerno tome koliko je sporiji od najbržeg u skupu, ispravilo je taj slučaj bez uklanjanja postojećih članova formule.

6.3 Objašnjenje predložene rute — slučaj Novi Banovci

Kada izabrana ruta odstupa od geometrijski najkraćeg puta, vozaču nije dovoljno prikazati *šta* je predloženo — potrebno mu je objašnjenje *zašto*. Paket `explain` implementira taj mehanizam prema principu binarne pretrage po dimenzijama profila vozila:

1. Zahteva se ”referentna” ruta sa svim dimenzijama profila veštački opuštenim na ne-realno velike vrednosti (npr. visina 100m, masa 900 000 kg) — ruta koju bi Valhalla predložila da fizička ograničenja vozila ne postoje.

2. Ako se izabrana ruta (sa stvarnim profilom vozila) po rastojanju ne razlikuje značajno od referentne (manje od 1km), nema odstupanja koje treba objasniti.
3. U suprotnom, dimenzije profila (visina, masa, osovinsko opterećenje, širina, prevoz opasnog tereta) se redom, jedna po jedna, "oslobađaju" na referentnu vrednost, i ruta se ponovo traži — dimenzija čije oslobađanje vrati rutu na rastojanje referentne rute je **vezujuće ograničenje** za taj segment puta.
4. Mesto odstupanja se određuje geometrijski: dekodirane geometrije (polyline) izabrane i referentne rute se upoređuju tačku po tačku, a prva tačka izabrane rute čija je udaljenost od svake tačke referentne rute veća od 200m proglašava se tačkom divergencije; njoj se pridružuje naziv ulice najbližeg manevra izabrane rute.
5. Generiše se poruka vozaču, npr.: *"Ruta skreće kod [ulica] jer visina vozila (4.7m) ne zadovoljava ograničenje na toj deonici."*



Slika 6.1: Tok mehanizma objašnjenja rute (explain.Explain)

Slučaj Novi Banovci. Ovaj mehanizam je razvijen i potvrđen na konkretnom, stvarnom slučaju: ručnom binarnom pretragom (ponavljanjem /route poziva sa različitim vrednosti-

ma visine) utvrđeno je da na auto-putu A1, u okolini čvora Novi Banovci (izlaz 21/22), postoji ograničenje visine između 4.5m i 4.6m. Za vozilo profila korišćenog u testu (visina 4.7m), Valhalla je korektno birala detour preko lokalnog puta umesto direktnog nastavka auto-putem — a mehanizam opisan u ovom poglavlju automatizuje upravo tu istu binarnu pretragu, koju je autor prvo sproveo ručno da bi potvrdio da je ponašanje Valhalla-e (i, posledično, sistema) tačno.

Bitna napomena o ovom mehanizmu je jedna poznata, dokumentovana greška otkrivena tokom razvoja i potom ispravljena: rana verzija je poredila listu manevara (`street_names`) izabrane i referentne rute **po pozicionom indeksu u nizu** (manevar broj i izabrane rute sa manevrom broj i referentne rute). Ovo je radilo dobro kada su rute delile zajednički početni deo puta i lokalno se razdvajale, ali je davalo pogrešnu (preraniju) lokaciju za vozila sa toliko strogim ograničenjem da Valhalla za njih bira **globalno drugačiju strategiju rute od samog početka** — tada indeks i ne odgovara istom mestu na putu kod obe rute. Zamena poređenja po indeksu geometrijskim poređenjem (korak 4 gore) rešila je ovaj problem, jer geometrija ne zavisi od toga kako su manevri indeksirani, već isključivo od toga gde se putevi fizički razilaze. Ovaj slučaj je vredan pomena u radu jer ilustruje opštu lekciju: kada se dva niza podataka (manevri dve različite rute) upoređuju da bi se pronašla tačka razlike, poređenje po **pozicionoj strukturi** (indeksu) je lažno pouzdano — tačno je samo dok se strukture podudaraju, a tiho daje pogrešan rezultat kada se globalno razlikuju; poređenje po **sadržaju** (u ovom slučaju, stvarnoj geografskoj geometriji) je robusnije jer ne pretpostavlja da strukture uopšte odgovaraju jedna drugoj.

6.4 Sopstvena implementacija pretrage puta nad ograničenim podgrafom

Kao samostalna algoritamska celina, odvojena od produkcionog puta (nije uvezana u `main.go` niti u REST API — koristi se isključivo kroz sopstvene testove), implementirana je sopstvena Dijkstra i A *pretraga najkraćeg puta koja radi direktno nad OSM XML podacima, bez ikakve zavisnosti od Valhalla-e*. Svrha ovog modula nije da zameni Valhallu u produkciji (poglavlje 5.2 objašnjava zašto bi reimplementacija efikasnog routing engine-a nad grafom cele države bila nerealan cilj za obim ovog rada), već da (a) demonstrira, kroz sopstveni, testiran kod, principe na kojima Valhalla interno počiva (isključivanje fizički nedopustivih grana, pretraga najkraćeg puta uz heuristiku), i (b) da posluži kao alat za evaluaciju i poređenje Dijkstra i A pristupa nad istim, realnim podacima (poglavlje 9.2), sa direktnim pristupom OSM tagovima koje obični Valhalla `/route` odgovor ne izlaže (poglavlje 6.1).

Graf i model podataka. Graf se učitava iz `.osm` (XML) ekstrakta funkcijom `LoadOSMXML`, koja parsira way-eve sa tagom `highway` u usmerene ivice (`Edge`), čuvajući po ivici dužinu, klasu puta (`RoadClass`), podlogu (`Surface`) i, ako postoje, ograničenja `maxheight/maxweight/hazmat` preuzeta sa way-a. Node-ovi koji imaju sopstveni `barrier`

tag sa `maxheight/maxweight` (npr. rampa niže visine na inače prohodnom putu) čuvaju se kao ograničenje na samom čvoru (`Node.MaxHeightM`), odvojeno od ograničenja na ivici — jer, kako je objašnjeno u poglavlju 2.4, OSM te dve vrste ograničenja zavodi na dva različita nivoa.

Funkcija cene. Cena prelaska ivice je njena dužina u metrima, uvećana za faktor lošije podloge (`unpaved`, `gravel`, `dirt`, `sett` — 30% penal) i skalirana faktorom preferencije klase puta (Tabela 6.1) — magistralni putevi i auto-putevi imaju blagi popust u ceni (manje raskrsnica/prilaza po kilometru za veliko vozilo), dok sekundarni putevi imaju blagi penal. Na svaki prelaz sa ivice na ivicu (skretanje) dodaje se i penal za oštrinu ugla skretanja, izražen u istoj jedinici (”metar”) kao i osnovna cena, radi jednostavnog sabiranja — svi ovi koeficijenti su, kao i kod paketa `scoring` (6.2), heuristička, nekalibrisana prva procena, što je u kodu izričito dokumentovano.

Tabela 6.1: Faktor preferencije klase puta
(`roadClassMultiplier`)

Klasa puta (OSM highway)	Faktor
<code>motorway</code>	0.85
<code>motorway_link</code>	0.90
<code>trunk</code>	0.90
<code>trunk_link</code>	0.95
<code>primary</code>	1.00
<code>primary_link</code>	1.05
<code>secondary</code>	1.15
<code>secondary_link</code>	1.20

Listing 6.4 prikazuje isključivanje nedopustivih ivica i čvorova (tvrdo ograničenje, princip identičan Valhalla-inom `truck costing`-u, poglavlje 2.3) i funkciju cene ivice.

```
func allowed(e Edge, p VehicleProfile) bool {
    if e.MaxHeightM > 0 && p.HeightM > e.MaxHeightM {
        return false
    }
    if e.MaxWeightT > 0 && p.WeightKg/1000 > e.MaxWeightT {
        return false
    }
    if e.Hazmat && p.Hazmat {
        return false
    }
    return true
}
```

```

}

func cost(e Edge) float64 {
    base := e.LengthM
    switch e.Surface {
    case "unpaved", "gravel", "dirt", "sett":
        base *= 1.3
    }
    if mult, ok := roadClassMultiplier[e.RoadClass]; ok {
        base *= mult
    }
    return base
}

```

Listing 6.4: Isključivanje nedopustivih ivica i funkcija cene (backend/internal/algorithm/cost.go)

Pretraga. Dijkstra i A *dele istu implementaciju pretrage (funkcija search)*, sa jedinom razlikom u heuristici prosleđenoj prioritetnom redu: Dijkstra koristi heuristiku koja je uvek nula (ekvivalentno standardnom Dijkstra algoritmu), a A koristi haversine udaljenost od trenutnog čvora do cilja. Pretraga koristi binarnu gomilu (container/heap) kao prioritetni red i prekida se čim je ciljani čvor skinut sa reda, umesto da izračuna najkraći put do svih čvorova u grafu. Listing 6.5 prikazuje deljenu funkciju pretrage.

```

func search(g *Graph, start, goal int64, profile VehicleProfile,
    heuristic func(node int64) float64) (Result, error) {
    dist := map[int64]float64{start: 0}
    prev := map[int64]int64{}
    pq := &priorityQueue{{node: start, priority: heuristic(start)}}
    heap.Init(pq)
    visited := map[int64]bool{}

    for pq.Len() > 0 {
        current := heap.Pop(pq).(pqItem).node
        if visited[current] {
            continue
        }
        visited[current] = true
        if current == goal {
            return Result{Path: reconstructPath(prev, start, goal), Cost: dist[goal]
                }, nil
        }
        for _, edge := range g.AdjList[current] {

```

```

    if !allowed(edge, profile) || !nodeAllowed(g.Nodes[edge.To], profile)
        || visited[edge.To] {
        continue
    }
    edgeCost := cost(edge)
    if p, ok := prev[current]; ok {
        edgeCost += turnPenaltyMeters(turnAngle(g, p, current, edge.To))
    }
    newDist := dist[current] + edgeCost
    if old, ok := dist[edge.To]; !ok || newDist < old {
        dist[edge.To] = newDist
        prev[edge.To] = current
        heap.Push(pq, pqItem{node: edge.To, priority: newDist + heuristic(
            edge.To)})
    }
}
}
return Result{}, errNoPath
}

```

Listing 6.5: Deljena implementacija Dijkstra i A* pretrage (backend/internal/algorithm/dijkstra.go)

Poznato, dokumentovano pojednostavljenje. Penal za skretanje se računa na osnovu **jednog** prethodno fiksiranog puta do trenutnog čvora (`prev[current]`), a ne na osnovu para (čvor, dolazna ivica), koje bi bilo teorijski korektnije (jer bi omogućilo da se do istog čvora stigne različitim uglovima dolaska i time različitim penalom za naredno skretanje). Ovo je jeftinije i jednostavnije za implementaciju od praćenja stanja pretrage po paru (čvor, ivica), po cenu da pretraga, u retkim slučajevima, ne mora biti globalno optimalna po ukupnom penalu za skretanje — pojednostavljenje koje je u samom kodu izričito i pošteno dokumentovano, u istom duhu kao i nekalibrisani koeficijenti pomenuti ranije u ovom poglavlju. Evaluacija ovog modula nad stvarnim podacima data je u poglavlju 9.2.

7 Backend servis i model podataka

7.1 REST API

Backend izlaže REST API organizovan oko pet grupa resursa: autentifikacija, vozila, ture (trips), dispečerski odnosi i chat. Tabela 7.1 sumira najvažnije endpoint-e; kompletan spisak obuhvata i dodatne varijante (npr. PATCH, PUT) koje ovde nisu posebno izdvojene.

Tabela 7.1: Pregled REST API endpoint-a (skraćeno)

Grupa	Endpoint	Opis
Autentifikacija	POST /auth/register, POST /auth/login, POST /auth/google	Registracija i prijava (lozinka ili Google)
Autentifikacija	GET /auth/verify-email, POST /auth/resent-verification	Verifikacija email adrese
Autentifikacija	POST /auth/forgot-password, GET/POST /auth/reset-password	Obnova zaboravljene lozinke
Autentifikacija	GET /auth/me, POST /auth/logout-all	Podaci o nalogu, odjava sa svih uređaja
Vozila	POST/GET/PUT/DELETE /vehicles	CRUD nad profilom vozila
Vozila	PATCH /vehicles/{id}/status, GET /vehicles/{id}/hours	Nivo goriva/servis, sati vožnje
Rutiranje	POST /api/v1/routes	Generisanje, rangiranje i objašnjenje rute (bez čuvanja)
Ture	POST/GET /trips, GET/PUT /trips/{id}	Kreiranje i pregled tura

Tabela 7.1: Pregled REST API endpoint-a (skraćeno)

Grupa	Endpoint	Opis
Ture	POST /trips/{id}/accept (i reject, start, position, reroute, complete)	Prelazi statusne mašine ture
Ture	GET /trips/{id}/events	Dnevnik događaja tokom vo- žnje
Dispečer	GET /dispatcher/drivers,	Pregled vozača dispečera / dostupnih vozača
	GET /dispatcher/available-drivers	
Dispečer	POST /dispatcher/requests,	Zahtev za povezivanje dispe- čer ↔ vozač
	GET /dispatcher/requests,	
	GET /driver/requests,	
	POST /driver/requests/{id}/respond	
Chat	GET /chats, GET/POST /chats/{driverId}/messages	Pregled i slanje poruka
Real-time	GET /ws/trips/{id}, GET /ws/chats/{counterpartId}	WebSocket veze (poglavlje 7.5)
Geokodiranje	GET /geocode, GET /geocode/reverse	Proxy ka Nominatim servisu

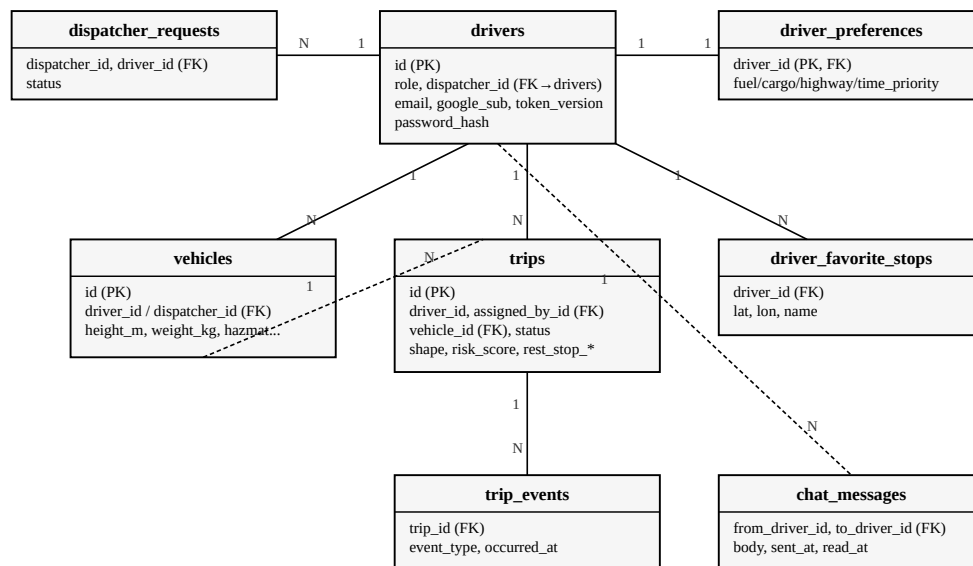
Endpoint POST /api/v1/routes je *stateless* — samo generiše i vraća pregled rute (uz rizik i objašnjenje), bez upisa u bazu — dok POST /trips kreira stvarnu turu koja prolazi kroz statusnu mašinu opisanu u nastavku. Ovo razdvajanje omogućava vozaču da pregleda više varijanti rute (npr. sa različitim profilom vozila ili preferencama) bez posledica u bazi, pre nego što se konkretna ruta konvertuje u turu.

Statusna mašina ture. Tura prolazi kroz stanja: offered (ponuđena vozaču od strane dispečera, čeka odgovor) → accepted (vozač prihvatio ponudu) / rejected (vozač odbio, terminalno stanje) → created (samostalno kreirana od strane vozača, bez posrednog offered stanja) → in_progress (vozač pokrenuo turu) → completed. Backend, preko HasActiveTrip/HasPendingOffer proveru, sprečava da vozač ima paralelno više aktivnih

tura ili više neodgovorenih ponuda istovremeno.

7.2 Model podataka

Slika 7.1 prikazuje pregled entiteta baze podataka i njihovih veza; potpuna šema, uključujući sve kolone, definisana je preko četiri goose migracije opisane u nastavku.



Isprekidane linije: veza posredna preko drivers (vlasništvo vozila / autor poruke), ne direktan FK prikazan na dijagramu.

Slika 7.1: Pregled entiteta baze podataka (skraćeno, bez svih kolona)

Šema baze je organizovana u četiri numerisane, hronološke migracije, upravljane alatom goose, koji garantuje da se svaka migracija primeni tačno jednom, bez obzira na to koliko puta se sistem pokrene. Tabela 7.2 sumira sadržaj svake migracije.

Tabela 7.2: Migracije šeme baze podataka

Migracija	Sadržaj
00001_initial_schema.sql	Osnovna šema: drivers, vehicles, trips, driver_preferences, driver_favorite_stops, trip_events, chat_messages

Tabela 7.2: Migracije šeme baze podataka

Migracija	Sadržaj
00002_dispatcher_roles.sql	drivers.role/dispatcher_id, vehicles.dispatcher_id, trips.assigned_by_id, tabela dispatcher_requests
00003_google_and_email_auth.sql	drivers.google_sub/email/email_verified, tabela email_verification_tokens
00004_password_reset_and_logout_all_drivers.sql	drivers.token_version, tabela password_reset_tokens

Listing 7.1 prikazuje deo osnovne šeme — tabele `vehicles` i `trips`, sa `CHECK` ograničenjem koje garantuje da osovinsko opterećenje vozila ne može biti veće od njegove ukupne mase (integritetno ograničenje na nivou baze, ne samo na nivou aplikacije).

```
CREATE TABLE IF NOT EXISTS vehicles (
  id SERIAL PRIMARY KEY,
  height_m DOUBLE PRECISION NOT NULL,
  width_m DOUBLE PRECISION NOT NULL,
  length_m DOUBLE PRECISION NOT NULL,
  weight_kg DOUBLE PRECISION NOT NULL,
  axle_load_kg DOUBLE PRECISION NOT NULL,
  hazmat BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (axle_load_kg <= weight_kg)
);

CREATE TABLE IF NOT EXISTS trips (
  id SERIAL PRIMARY KEY,
  vehicle_id INTEGER NOT NULL REFERENCES vehicles(id),
  origin_lat DOUBLE PRECISION NOT NULL,
  origin_lon DOUBLE PRECISION NOT NULL,
  destination_lat DOUBLE PRECISION NOT NULL,
  destination_lon DOUBLE PRECISION NOT NULL,
  distance_km DOUBLE PRECISION NOT NULL,
  duration_min DOUBLE PRECISION NOT NULL,
  risk_score DOUBLE PRECISION NOT NULL,
  shape TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'created',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);
```

Listing 7.1: Osnovna šema tabela `vehicles` i `trips` (`00001_initial_schema.sql`)

Geometrija rute (`trips.shape`) čuva se kao enkodovan tekstualni polyline (isti format koji vraća Valhalla), a ne kao PostGIS `geometry` kolona — pragmatičan izbor koji izbegava dekodiranje/enkodiranje geometrije pri svakom čitanju iz baze, po cenu da prostorni upiti nad geometrijom rute (npr. "sve ture koje prolaze kroz ovaj region") nisu mogući direktno u bazi, već bi zahtevali dekodiranje u aplikacionom sloju — razmatrano kao mogući pravac budućeg rada u poglavlju 10.

7.3 Uloge vozač/dispečer

Sistem razlikuje dve uloge vozača: **vozač** (`driver`) i **dispečer** (`dispatcher`), zapisane u koloni `drivers.role`. Veza dispečer ↔ vozač se, namerno, ne uspostavlja pri registraciji, već isključivo preko mehanizma zahteva i odobrenja: dispečer šalje zahtev (`POST /dispatcher/requests`) izabranom vozaču, a vozač ga prihvata ili odbija (`POST /driver/requests/{id}/respond`); prihvatanje postavlja `drivers.dispatcher_id` na identifikator dispečera. Vozač u svakom trenutku može napustiti dispečera (`POST /driver/leave-dispatcher`), čime se ta veza raskida.

Vozilo (`vehicles`) pripada tačno jednom od dva vlasnika — `driver_id` ili `dispatcher_id` — nikad oboma. Ovim se modeluje realan scenario iz transportne prakse: vozač koji radi za dispečera može voziti vozilo koje je u vlasništvu firme (dispečera), ali isto tako može, paralelno, imati i sopstveno vozilo kojim upravlja samostalno — provera koje vozilo vozač trenutno može koristiti (`vehicleAccessible/vehicleMutable`) implementirana je na nivou Go handler-a, a ne kao ograničenje baze, jer zavisi i od trenutne dispečer-vozač veze, koja se može promeniti tokom vremena.

Kada dispečer kreira turu za vozača (`assigned_by_id` postavljen na dispečera), ruta se generiše i rangira prema **dispečerovom** profilu preferenci, ne vozačevom — s obzirom na to da je dispečer taj koji donosi odluku o ruti u ime firme, a preference u ovom sistemu modeluju stil odlučivanja o ruti, ne lične navike vozača u vožnji.

7.4 Asinhrona obrada — RabbitMQ i modul pauza vozača

Kada vozač pokrene turu, backend objavljuje `trip.started` poruku na `trip.events` topic exchange RabbitMQ-a. Nezavisan worker proces, pokrenut kao goroutine unutar istog backend binarnog fajla (ne kao poseban kontejner), konzumira tu poruku i računa jednostavan predlog pauze: ako je planirano trajanje ture veće od praga od 270 minuta (4.5h — pojednostavljena zamena za stvarno AETR pravilo o obaveznoj pauzi nakon određenog vremena vožnje [18], obrazloženo dalje u ovom poglavlju), worker pronalazi tačku na ruti gde bi

se vozilo teorijski nalazilo nakon isteka tog vremena (pod pretpostavkom konstantne brzine duž rute), i traži najbliže odmaralište iz skupa OSM node-ova učitanih u poglavlju 5.3 koje je stvarno u koridoru trase. Listing 7.2 prikazuje deo implementacije koja računa predlog.

```
const restThresholdMin = 270 // 4.5h - stand-in za AETR pravilo

func (w *TripWorker) computeRestStop(ctx context.Context, trip store.Trip)
    store.RestStopSuggestion {
    if trip.DurationMin <= restThresholdMin {
        return store.RestStopSuggestion{}
    }
    afterMin := float64(restThresholdMin)
    suggestion := store.RestStopSuggestion{AfterMinutes: &afterMin}

    points := valhalla.DecodePolyline6(trip.Shape)
    fraction := restThresholdMin / trip.DurationMin
    at := valhalla.PointAtFraction(points, fraction)

    // ... čitavanje preferenci čvozaa i omiljenih stanica ...
    stop, _, found := w.RestStops.NearestOnRoute(at.Lat, at.Lon, brand,
        favorites,
        reststop.DefaultPreferredRadiusM, routePoints, reststop.
            DefaultRouteCorridorRadiusM, hazmat)
    if !found {
        return suggestion
    }
    suggestion.Lat, suggestion.Lon = &stop.Lat, &stop.Lon
    suggestion.Amenity = &stop.Amenity
    return suggestion
}
```

Listing 7.2: Proračun predloga pauze u TripWorker (backend/internal/worker/trip_worker.go)

Rezultat se upisuje u samu turu (trips.rest_stop_* kolone) i u dnevnik događaja (trip_events, tip rest_stop_suggested), koji mobilna aplikacija prikazuje vozaču kao vremensku liniju (poglavlje 8.1). Za vozila koja prevoze opasan teret, pretraga preferira benzinske stanice nad parkinzima (u granicama tolerancije rastojanja), a rezultat u svim slučajevima favorizuje vozačevu omiljenu ili brend-specifičnu stanicu, ako se takva nalazi u koridoru trase.

Namerno pojednostavljenje. Prag od 270 minuta je jednostavna, fiksna zamena za stvarnu evropsku AETR regulativu o radnom vremenu i obaveznim pauzama vozača teretnih

vozila [18], koja u stvarnosti razlikuje dnevno i nedeljno vreme vožnje, obavezne dnevne i nedeljne odmone, i dozvoljava određena izuzeća. Puna implementacija te regulative izlazi iz obima ovog rada (obrazloženo u poglavlju 10) — cilj modula je da demonstrira **mehanizam** (asinhrono računanje predloga na osnovu proteklog vremena vožnje i geografske pretrage najbliže odgovarajuće lokacije), a ne da zameni stvarnu, zakonski obavezujuću logiku.

Napomena o evoluciji sistema. U ranoj fazi razvoja bio je planiran i drugi tok poruka — `trip.eta_updated`, koji bi worker objavljivao nakon računanja ETA, a WebSocket gateway (7.5) konzumirao radi guranja ka klijentu. Taj tok je napušten kada je uvedeno praćenje stvarne GPS pozicije vozila (umesto simulacije), pošto je ETA tada postalo jednostavnije računati direktno u WebSocket gateway-u, iz svake nove GPS pozicije (poglavlje 7.5) — čime je `trip.eta_updated` poruka postala "mrtav kod" (niko je nije konzumirao) i uklonjena je iz sistema. Ovo je uobičajena i zdrava evolucija u razvoju asinhronog sistema: mehanizam koji je u jednom trenutku bio neophodan postaje suvišan kada se promeni pretpostavka na kojoj je zasnovan, i treba ga ukloniti umesto ostaviti kao neaktivan kod.

7.5 Real-time komunikacija — WebSocket gateway

Paket `ws` implementira dva WebSocket gateway-a nad istom osnovnom idejom: relej poruka svim zainteresovanim klijentima jedne "teme" (ture ili konverzacije), bez čuvanja stanja konverzacije unutar samog WebSocket sloja.

Praćenje pozicije uživo. Za svaku turu koja je u toku, gateway drži strukturu `liveTrip` sa skupom pretplatnika (otvorenih WebSocket konekcija) i poslednjom emitovanom pozicijom. Kada vozačev telefon prijavi novu GPS poziciju (`POST /api/v1/trips/{id}/position`), gateway izračunava preostalo rastojanje do odredišta (haversine formula) i procenjeno vreme dolaska na osnovu prosečnog planiranog tempa ture, i tu poruku emituje svim trenutno povezanim pretplatnicima — vozačevom sopstvenom ekranu i, ako postoji, njegovom dispečeru koji istu turu prati na svojoj live mapi (poglavlje 8.2). Novi pretplatnik koji se poveže (ili ponovo poveže) odmah prima poslednju poznatu poziciju, umesto da čeka narednu GPS prijavu — bez ovoga, dispečer koji zatvori i ponovo otvori live mapu ne bi video ništa do naredne GPS prijave vozila, koja može kasniti minutima ako se vozilo ne kreće (GPS prijave se šalju na pomak od određenog broja metara, ne na fiksni interval).

Napomena o evoluciji sistema. Prva verzija ovog gateway-a je, kada još nije stigla nijedna stvarna GPS prijava, simulirala kretanje vozila "hodanjem" duž geometrije rute fiksnim korakom, kao zamenu za stvarni GPS signal tokom razvoja bez pristupa fizičkom vozilu. Kada je testiranje sa stvarnim telefonom postalo dostupno, simulacija je počela da pravi vidljiv "skok" na ekranu u trenutku kada bi stigla prva stvarna GPS prijava (jer se stvarna pozicija po pravilu nije poklapala sa simuliranom) — problem koji je rešen potpunim uklanjanjem simulacije, pošto je do tog trenutka mobilna aplikacija već zahtevala stvaran GPS fix pre ne-

go što dozvoli pokretanje ture (provera blizine polazištu, poglavlje 8.1), čime je simulacija izgubila i svoju originalnu svrhu.

Chat. Chat poruke se, za razliku od pozicije, trajno čuvaju u bazi (`chat_messages`) preko REST endpoint-a (`POST /chats/{driverId}/messages`) — REST je "izvor istine" za istoriju razgovora. WebSocket veza (`GET /ws/chats/{counterpartId}`) služi isključivo za isporuku uživo dok je druga strana trenutno povezana: backend objavljuje poslatu poruku na RabbitMQ (`chat.events` exchange), a WS gateway je samo prosleđuje pretplatniku ako je u tom trenutku povezan — ako nije, poruka je već sačuvana u bazi preko REST poziva i biće učitana pri narednom otvaranju razgovora, bez potrebe za posebnim mehanizmom "neisporučenih" poruka.

8 Mobilna aplikacija

Mobilna aplikacija je jedinstvena Flutter kodna baza koja se, u zavisnosti od uloge prijavljenog korisnika (poglavlje 7.3), grana na dva različita korisnička toka: vozača i dispečera. Grananje se vrši na jednom mestu (`entry_router`), odmah nakon prijave, na osnovu polja `role` vraćenog sa backend-a.

8.1 Tok korisnika — vozač

Nakon prijave (`login_screen`, `register_screen` sa izborom uloge vozač/dispečer preko `SegmentedButton`-a, ili `forgot_password_screen` za obnovu lozinke), samostalni vozač ili vozač koji radi za dispečera dolazi na svoj početni ekran. Ako je trenutno pod dispečerom i čeka na ponudenu turu, prikazuje mu se `offered_trips_screen` sa ponudama koje treba prihvatiti ili odbiti; u suprotnom, vozač sam pokreće planiranje preko `route_request_screen`.

Planiranje rute. `route_request_screen` omogućava izbor polazišta (trenutna GPS pozicija ili pretraga adrese preko `address_search_field`, koja poziva backend `GET /geocode proxy` ka Nominatim-u) i odredišta, izbor vozila (preko `vehicle_list_screen/vehicle_profile_screen`, gde vozač unosi ili menja visinu, širinu, dužinu, masu, osovinsko opterećenje i prevoz opasnog tereta), i prikaz vraćenih alternativnih ruta na mapi, sortiranih po riziku (poglavlje 6.2), sa mogućim objašnjenjem odstupanja rute (poglavlje 6.3) prikazanim kao tekstualna napomena.

Pokretanje i praćenje ture. Pre pokretanja ture, `start_proximity_status` widget proverava da je vozač stvarno u blizini (unutar 500m) prijavljenog polazišta preko GPS-a — namerna provera koja sprečava pokretanje ture "iz kancelarije" i garantuje da je prva pozicija koju backend primi zaista stvaran GPS fix, ne proizvoljna vrednost (relevantno za odluku da se ukloni simulacija pozicije, poglavlje 7.5). Tokom vožnje, `active_trip_screen` prikazuje trenutnu poziciju, procenjeno vreme dolaska i alert za predlog pauze (poglavlje 7.4) primljene preko WebSocket veze (`trip_socket.dart`, sa automatskim ponovnim povezivanjem po rastućem intervalu — 1, 2, 5, potom 10 sekundi — u slučaju privremenog prekida konekcije), a `active_trip_banner` ostaje vidljiv i na drugim ekranima aplikacije dok je tura aktivna, kao podsetnik i prečica nazad na `active_trip_screen`. Ako vozač napusti planiranu rutu za više od 300m, backend automatski preračunava novu rutu od trenutne pozicije do istog odredišta (`POST /trips/{id}/reroute`).

Dnevnik i istorija. `trip_log_screen` prikazuje dnevnik događaja jedne ture (`trip_events` — polazak, predlog pauze, dolazak) kao vremensku liniju, dok `trip_list_screen` organizuje sve vozačeve ture u tri kategorije (u toku, predstojeće, zavr-

šene). `truck_status_screen` prikazuje nivo goriva, preostali kilometre do sledećeg servisa i ukupne sate vožnje vozila; `preferences_screen` omogućava podešavanje stila vožnje (klizači za prioritet potrošnje goriva, osetljivosti tereta, korišćenja auto-puta i brzine, opisani u poglavlju 6.2) i omiljene benzinske stanice; `chat_list_screen/chat_thread_screen` omogućavaju dopisivanje sa dispečerom u realnom vremenu (poglavlje 7.5).

8.2 Tok korisnika — dispečer

Dispečer upravlja flotom vozača i vozila kroz odvojen skup ekrana. `dispatcher_home_screen` je početna tačka; `dispatcher_available_drivers_screen` prikazuje vozače koji trenutno nisu vezani za nijednog dispečera i omogućava slanje zahteva za povezivanje (poglavlje 7.3), dok `dispatcher_requests_screen` prikazuje status poslatih zahteva. `dispatcher_create_trip_screen` omogućava kreiranje i dodelu ture jednom od svojih vozača, birajući između vozačevog sopstvenog vozila ili vozila u vlasništvu dispečera (poglavlje 7.3).

Najkompleksniji ekran dispečerskog toka je `dispatcher_live_map_screen`: prikazuje jednu mapu sa **N paralelnih WebSocket konekcija**, po jednu za svaku aktivnu turu svojih vozača, i ažurira poziciju svakog vozila na mapi čim gateway (poglavlje 7.5) emituje novu poziciju za odgovarajuću turu. `dispatcher_trip_list_screen` prikazuje istoriju svih tura koje je dispečer dodelio.

8.3 Arhitektura mobilne aplikacije

Aplikacija je organizovana u standardnu Flutter strukturu: `screens/` (ekrani navedeni u 8.1/8.2), `models/` (podatkovne klase koje odgovaraju backend JSON odgovorima — `driver`, `vehicle_profile`, `trip`, `trip_event`, `route_result/route_candidate`, `chat_message`, `dispatcher_request`, `driver_preferences`, `favorite_stop`, `geocode_result`, `position_update`, `rest_stop`, `account_status`, `auth_result`), `services/` i `widgets/` (deljene, ponovo iskoristive komponente).

Sloj servisa uključuje: `api_client.dart` (centralizovan REST klijent koji, pored slanja zahteva, globalno presreće HTTP 401 odgovor i automatski prebacuje korisnika na ekran za prijavu, umesto da svaki ekran posebno rukuje isticanjem sesije), `auth_storage.dart` (lokalno, trajno čuvanje JWT tokena na uređaju), `chat_socket.dart` i `trip_socket.dart` (WebSocket klijenti sa automatskim ponovnim povezivanjem), `location_service.dart` (očitanje GPS pozicije preko `geolocator` biblioteke), `google_auth.dart` (Google prijava na klijentskoj strani), `polyline.dart` (dekodiranje enkodovane geometrije rute za prikaz na mapi) i `route_observer.dart` (Flutter `RouteAware` mehanizam koji osvežava podatke na ekranu kada se korisnik na njega vrati nakon zatvaranja nekog drugog ekrana — npr. povratak na listu vozila nakon uređivanja profila vozila).

Među deljenim widgetima ističe se `radial_fab_menu.dart` — radijalni, povlačivi (*draggable*) meni pokrenut iz plutajućeg dugmeta (*floating action button*), korišćen kao glavna navigacija umesto klasične donje trake sa karticama (*bottom navigation bar*), kao i `email_verification_banner` (podsetnik da nalog nije verifikovan) i `start_proximity_status` (opisan u 8.1).

9 Testiranje i evaluacija

9.1 Automatizovano testiranje backend-a

Backend je testiran automatizovanim, tabelarno-vođenim (*table-driven*) Go testovima, standardnim pristupom u Go ekosistemu gde se više varijacija istog scenarija testira kroz jednu test funkciju sa nizom ulaznih/izlaznih parova. Tabela 9.1 prikazuje broj test funkcija po paketu, na dan pisanja rada; svi navedeni testovi prolaze (`go test ./...` vraća ok za svaki paket koji sadrži testove).

Tabela 9.1: Broj automatizovanih testova po Go paketu

Paket	Broj testova	Fokus testiranja
internal/algorithm	15	Isključivanje nedopustivih ivica/čvorova, poklapanje Dijkstra/A*, penal za skretanje, ponašanje nad realnim OSM podacima
internal/scoring	6	Funkcija bodovanja rute, preferenca korisnika, popust za omiljenu stanicu
internal/explain	8	Detekcija odstupanja rute, identifikacija vezujuće dimenzije, geometrijska divergencija
internal/reststop	9	Pretraga najbližeg odmara lišta u koridoru rute, hazmat preferenca, brend/omiljena lokacija
internal/auth	5	Izdavanje/parsiranje JWT tokena, provera lozinke, verzija tokena
internal/ws	4	Relej pozicije pretplatnicima, replay poslednje pozicije novom pretplatniku
internal/httpapi	4	HTTP handleri (integracioni testovi nad REST rutama)

Tabela 9.1: Broj automatizovanih testova po Go paketu

Paket	Broj testova	Fokus testiranja
internal/geocode	6	Nominativ proxy, throttling
Ukupno	57	

Paket `internal/algorithm` koristi dve vrste test podataka: (a) male, ručno konstruisane sintetičke grafove (npr. "dijamant" od četiri čvora sa kontrolisanim ograničenjima), pogodne za preciznu proveru jedne izolovane osobine algoritma (npr. "grana sa `maxheight=3.5` se isključuje za vozilo visine 4.0m"), i (b) stvaran OSM ekstrakt koridora Beograd–Novi Sad (poglavlje 9.2), koji dodatno potvrđuje da implementacija korektno parsira i koristi *stvarne* OSM tagove, ne samo tagove koje je autor sam smislio za potrebe testa.

9.2 Evaluacija sopstvenog algoritma nad realnim podacima

Sopstvena Dijkstra/A* implementacija (poglavlje 6.4) je evaluirana nad stvarnim OSM ekstraktom koridora Beograd–Novi Sad (major-roads-only ekstrakt, `testdata/beograd-novisad-corridor` učitanim istim `LoadOSMXML` mehanizmom koji koristi i produkcionni kod. Ekstrakt se učitava u graf od **38 265 čvorova i 50 705 usmerenih ivica** — dva reda veličine manje od potpunog nacionalnog Valhalla grafa (898 337 čvorova, poglavlje 5.2), u skladu sa namerom da ovaj modul bude ograničen, transparentan alat za evaluaciju, ne zamena za Valhallu.

Plauzibilnost rute. Za par tačaka Beograd (44.80, 20.40) – Novi Sad (45.25, 19.85), Dijkstra pretraga (vozilo visine 4.0m, mase 40 000 kg) pronalazi put od **81,0 km kroz 1899 čvorova**. Za poređenje, Valhalla, nad kompletnim nacionalnim grafom sa punim skupom puteva (ne samo magistralnim), za isti par tačaka vraća rute u rasponu od 84,8 do 92,7 km (u zavisnosti od izabrane alternative), zabeleženo tokom rada na Valhalla integraciji (poglavlje 6.1). Rezultat sopstvenog modula je unutar plauzibilnog opsega — blago kraći, što je i očekivano jer ograničeni ekstrakt sadrži samo magistralne puteve i ne modeluje sve raskrsnice/petlje koje Valhalla uzima u obzir sa kompletnim grafom, a start/cilj pretrage su najbliži čvorovi grafa vazdušnoj liniji do date koordinate, ne tačno iste ulazno-izlazne rampe koje bi Valhalla izabrala.

Isključivanje na osnovu stvarnog OSM tag-a. Nad istim ekstraktom pronađena je stvarna deonica sa tagom `maxheight=4.3` u centralnoj zoni Beograda. Za vozilo visine 4.0m, Dijkstra pronalazi put od 4036m preko te deonice; za vozilo visine 4.5m (koje premašuje 4,3m ograničenje), pretraga korektno ne pronalazi nijedan put između istih dveju tačaka — u ovom ograničenom, samo-magistralnom ekstraktu ne postoji alternativna deonica koja bi vozilo moglo koristiti umesto isključene, pa je ishod potpuno odsustvo puta, a ne duži za-

obilazni put. Ovaj test potvrđuje da mehanizam isključivanja radi korektno na **stvarnim**, ne samo ručno konstruisanim OSM podacima.

Poklapanje Dijkstra i A*. Za isti par (Beograd, Novi Sad) i profil, Dijkstra i A implementacija vraćaju identičnu cenu puta, što je i teorijski očekivano za dopustivu heuristiku (poglavlje 2.3) — A* ne pronalazi drugačiji (a kamoli lošiji) put, samo ga pronalazi pregledavajući manji broj čvorova. Vremenska analiza (Tabela 9.2) potvrđuje da je pretraga, nad grafom ove veličine, brza u odnosu na jednokratno parsiranje ulaznog XML fajla — 11,3 MB OSM XML-a.

Tabela 9.2: Vreme izvršavanja nad koridorom Beograd–Novi Sad (38 265 čvorova, 50 705 ivica)

Korak	Vreme
Učitavanje i parsiranje OSM XML ekstrakta (LoadOSMXML)	≈ 405 ms
Dijkstra pretraga (Beograd → Novi Sad)	≈ 19,3 ms
A* pretraga (Beograd → Novi Sad)	≈ 17,1 ms

Rezultat u Tabeli 9.2 potvrđuje očekivanje iz poglavlja 2.3 i 6.4: sama pretraga, jednom kada je graf učitao u memoriju, izvršava se za desetine milisekundi i dominantno je ograničena parsiranjem ulaznih podataka, a ne samim algoritmom pretrage — što je i razlog zbog kojeg je u produkcionom putu ovog sistema izgradnja i održavanje grafa u celosti prepušteno Valhalla-i (poglavlje 5.2, 6.1), koja tu istu vrstu posla radi jednom, unapred, za ceo nacionalni graf, umesto pri svakom zahtevu.

9.3 Uticaj prilagođene funkcije rizika — studija slučaja

Doprinos sopstvenog sloja rangiranja (poglavlje 6.2) u odnosu na "sirov" izbor prve Valhalla alternative ilustrovan je konkretnim slučajem otkrivenim tokom testiranja, na ruti u okolini mesta Radalj. Rana verzija formule bodovanja, koja nije sadržala vremenski član (timeTerm), birala je kao "najbolju" alternativu rutu koja je bila **47% duža i 30% sporija** od druge ponuđene alternative, isključivo zato što je imala povoljniji odnos magistralne i lokalne deonice puta (highwayTerm). Sa stanovišta stvarnog transportnog troška (vreme, gorivo, habanje vozila), taj izbor je bio pogrešan — kraća/brža alternativa je bila objektivno bolji izbor uprkos manje povoljnom highwayRatio.

Dodavanjem timeTerm-a, koji direktno kažnjava kandidata srazmerno njegovom relativnom kašnjenju u odnosu na najbrži kandidat u istom skupu alternativa (poglavlje 6.2, Listing 6.3), formula je počela korektno da favorizuje bržu/kraću rutu u ovom slučaju, bez uklanjanja postojećih članova formule (broj manevara, udeo magistralne deonice, potrošnja, oštri ma-

nevri) koji ostaju relevantni u slučajevima kada razlika u trajanju nije toliko izražena. Ovaj slučaj je dobar primer opšte lekcije o dizajnu heurističkih funkcija cene sa više članova: davanje novog signala treba motivisati **konkretnim, učenim lošim ishodom** postojeće formule, a ne unapred pretpostavljenom kompletnošću — isti princip po kome je i ova formula, i funkcija cene u poglavlju 6.4, iz istog razloga ostala eksplicitno obeležena kao nekalibrisana, prva heuristička procena, a ne konačan, empirijski potvrđen model.

9.4 Ograničenja identifikovana testiranjem

Testiranje je otkrilo i granice sistema koje nisu greške u uobičajenom smislu, već posledice svesno odabranog obima rada:

- Sopstveni Dijkstra/A* modul (poglavlje 6.4), primenjen na isti par koordinata za koji je poznat slučaj Novi Banovci (poglavlje 6.3), **ne reprodukuje** taj slučaj — u ograničenom, samo-magistralnom ekstraktu korišćenom za ovaj modul, u okolini Novih Banovaca ne postoji `maxheight` tag na way nivou. Najverovatnije objašnjenje, dokumentovano tokom razvoja, je da je stvarno ograničenje u tom slučaju zavedeno kao tačkasta (node) prepreka, ne kao way tag, i/ili da geometrijski najkraći put u ograničenom grafu jednostavno ne prolazi kroz tačno tu deonicu koju bi Valhalla, sa svojim potpuno drugačijim, kompletnim grafom i sopstvenom heuristikom, izabrala. Ovo je pošteno prijavljen negativan nalaz, ne prikriven — dodatno naglašava razliku u nameni dva sloja rutiranja u ovom sistemu: Valhalla (sa Explain mehanizmom, poglavlje 6.3) je taj koji stvarno rešava slučaj Novi Banovci u produkciji; sopstveni modul (6.4) demonstrira princip mehanizma isključivanja nad drugim, ali i dalje stvarnim, primerom istog tipa OSM ograničenja (`maxheight=4,3` u centralnom Beogradu, poglavlje 9.2).
- Penal za skretanje u sopstvenoj pretrazi (6.4) je, kako je i dokumentovano u samom kodu, teorijsko pojednostavljenje koje u retkim slučajevima ne garantuje globalno optimalan put po ukupnom penalu za skretanje.
- Funkcija rizika (6.2) i funkcija cene sopstvene pretrage (6.4) koriste bazne koeficijente koji su prva heuristička procena, ne kalibrisani prema stvarnim podacima o potrošnji goriva, habanju vozila ili statistici saobraćajnih nezgoda — razlika u odnosu na produkcioni sistem bi bila prikupljanje takvih podataka i kalibracija koeficijenata regresijom ili sličnim metodom, što izlazi iz obima ovog rada.

10 Zaključak i budući rad

Ovaj rad je pokazao da se prihvatljivo tačan, transparentan i **objašnjiv** sistem za rutiranje teretnih vozila može izgraditi nad potpuno otvorenim geografskim podacima (OpenStreet-Map) i otvorenim routing engine-om (Valhalla), uz sopstveni algoritamski doprinos koji tim alatima nedostaje. Implementiran je kompletan sistem koji obuhvata: pripremu geografskih podataka specifičnih za teretni saobraćaj (poglavlje 5), backend servis koji generiše, rangira sopstvenom funkcijom rizika i objašnjava predloženu rutu (poglavlje 6 i 7), sopstvenu, nezavisno testiranu Dijkstra/A* implementaciju nad ograničenim, ali stvarnim podacima (poglavlje 6.4, evaluirana u poglavlju 9.2), distribuiranu komunikaciju preko REST API-ja, RabbitMQ posrednika poruka i WebSocket veza (poglavlje 7), i mobilnu aplikaciju za dve različite korisničke uloge — vozača i dispečera (poglavlje 8). Konkretna slučaj (visinsko ograničenje kod Novih Banovaca, poglavlje 6.3) je korišćen kao stvaran, provaljen dokaz da mehanizam objašnjenja rute radi na pravom, a ne samo hipotetičkom problemu, a slučaj Radalj (poglavlje 9.3) kao konkretan primer u kome je sopstvena funkcija rizika ispravila objektivno pogrešan izbor rute.

10.1 Doprinosi rada

Glavni doprinosi rada su:

1. Sopstveni sloj rangiranja alternativnih ruta prema riziku specifičnom za teretni saobraćaj i preferencama vozača, nad Valhalla alternativama koje engine sam ne rangira (poglavlje 6.2).
2. Automatizovan mehanizam koji vozaču objašnjava zašto predložena ruta odstupa od geometrijski najkraćeg puta, sa geometrijskom (ne pozicionom) detekcijom mesta odstupanja, potvrđen na stvarnom slučaju (poglavlje 6.3).
3. Samostalna, testirana Dijkstra/A* implementacija nad ograničenim podgrafom OSM podataka, sa sopstvenom funkcijom cene i isključivanjem ivica/čvorova na osnovu fizičkih ograničenja vozila, evaluirana nad stvarnim, a ne samo sintetičkim podacima (poglavlje 6.4, 9.2).
4. Kompletan, iako po obimu ograničen, distribuiran sistem — REST API, asinhrona obrada porukama, WebSocket komunikacija u realnom vremenu — sa modelom uloga dispečer/vozač i mehanizmom ponude/prihvatanja ture (poglavlje 7).

10.2 Ograničenja rada

U skladu sa realno postavljenim obimom (obrazloženim već u poglavlju 1.2), rad svesno ne obuhvata:

- **Punu AETR regulativu** o radnom vremenu i pauzama vozača (poglavlje 7.4) — trenutni prag od 270 minuta je pojednostavljena zamena, ne zakonski obavezujuća logika.
- **Elevacioni (nagibni) rizik rute** — sistem nema pristup podacima o nagibu puta, pa je procena potrošnje goriva (poglavlje 6.2) relativna, ne apsolutna veličina.
- **Offline režim rada mobilne aplikacije** — aplikacija zahteva internet konekciju za svaku operaciju; rad bez signala/na lokalnoj mreži nije podržan.
- **Podršku za više država** — geografski podaci i pripremljen graf pokrivaju isključivo teritoriju Republike Srbije.
- **Produkcioni nivo bezbednosti** — WebSocket CheckOrigin je propustljiv (poglavlje 4.3), a RabbitMQ topologija je minimalna (jedan topic exchange, bez retry/DLQ mehanizma), prihvatljivo za sistem sa jednim poznatim klijentom, ne za javni, višezakupčani servis.
- **Kalibraciju koeficijenata funkcije rizika i funkcije cene** prema stvarnim empirijskim podacima (poglavlje 9.4) — koeficijenti su prva heuristička procena.
- **Prostorne upite nad geometrijom rute direktno u bazi** — geometrija se čuva kao tekstualni polyline, ne PostGIS geometry kolona (poglavlje 7.2), iako je PostGIS ekstenzija već deo infrastrukture.

10.3 Pravci budućeg rada

Na osnovu ograničenja iz 10.2, prirodni pravci budućeg razvoja su: (1) implementacija pune AETR logike, uključujući razliku između dnevnog i nedeljnog vremena vožnje i zakonski propisanih izuzetaka; (2) uvođenje elevacionih podataka (npr. iz SRTM ili sličnog izvora) u funkciju rizika, radi realnije procene potrošnje goriva na deonicama sa izraženim nagibom; (3) offline režim mobilne aplikacije sa lokalnim kešom poslednje poznate rute i odloženim slanjem GPS pozicija po povratku konekcije; (4) proširenje geografskog obuhvata na region ili celu Evropu, što bi zahtevalo i preispitivanje da li ograničeni, u memoriji učitani skup odmarališta (poglavlje 5.3) i dalje skalira, ili treba preći na PostGIS prostorni upit nad već postojećom ekstenzijom; (5) prikupljanje stvarnih podataka (npr. iz simulacije ili pilot upotrebe) radi kalibracije koeficijenata funkcije rizika (6.2) i funkcije cene sopstvenog algoritma (6.4) regresijom ili sličnim metodom, umesto ručno pretpostavljenih vrednosti; (6) bezbednosni hardening WebSocket sloja (stroža CheckOrigin provera, autentifikacija na nivou pojedinačne poruke) i proizvodna RabbitMQ topologija (retry, dead-letter queue) ako

bi sistem prešao iz demonstracionog u produkcionu kontekst sa nepoznatim, nepoverljivim klijentima.

Literatura

- [1] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [2] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*. Pearson, 3 ed., 2017.
- [3] I. Fette and A. Melnikov, “The WebSocket protocol.” RFC 6455, IETF, 2011.
- [4] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [5] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [6] OpenStreetMap Wiki, “Map features.” https://wiki.openstreetmap.org/wiki/Map_features, 2026. pristupljeno avgusta 2026.
- [7] Google, “The go programming language.” <https://go.dev>, 2026.
- [8] Google, “Flutter — build apps for any screen.” <https://docs.flutter.dev>, 2026.
- [9] “Valhalla routing engine.” <https://valhalla.github.io/valhalla/>, 2026.
- [10] “Postgis — spatial and geographic objects for postgresql.” <https://postgis.net>, 2026.
- [11] “Rabbitmq documentation.” <https://www.rabbitmq.com/documentation.html>, 2026.
- [12] “Docker compose overview.” <https://docs.docker.com/compose/>, 2026.
- [13] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT).” RFC 7519, IETF, 2015.
- [14] “Nominatim — openstreetmap geocoding.” <https://nominatim.org>, 2026.
- [15] “goose — database migration tool.” <https://github.com/pressly/goose>, 2026.
- [16] Geofabrik GmbH, “Openstreetmap data extracts.” <https://download.geofabrik.de>, 2026. pristupljeno 2026.

- [17] “osmium-tool — command line tool for working with openstreetmap data.” <https://osmcode.org/osmium-tool/>, 2026.
- [18] Evropski parlament i Savet, “Uredba (ez) br. 561/2006 o usklađivanju određenih socijalnih propisa u vezi sa drumskim saobraćajem (aetr),” 2006.

Spisak skraćenica

Skraćenica	Značenje
AETR	Evropski sporazum o radu posade vozila koja obavljaju međunarodne drumske prevoze (<i>Accord européen sur les Transports Routiers</i>)
AMQP	<i>Advanced Message Queuing Protocol</i>
API	<i>Application Programming Interface</i>
CRUD	<i>Create, Read, Update, Delete</i>
ETA	<i>Estimated Time of Arrival</i> — procenjeno vreme dolaska
GPS	<i>Global Positioning System</i>
HGV	<i>Heavy Goods Vehicle</i> — teško teretno vozilo
HTTP	<i>Hypertext Transfer Protocol</i>
JSON	<i>JavaScript Object Notation</i>
JWT	<i>JSON Web Token</i>
JWKS	<i>JSON Web Key Set</i>
MVP	<i>Minimum Viable Product</i>
OSM	<i>OpenStreetMap</i>
REST	<i>Representational State Transfer</i>
SMTP	<i>Simple Mail Transfer Protocol</i>
SQL	<i>Structured Query Language</i>
VRP	<i>Vehicle Routing Problem</i>
WS	WebSocket

Spisak slika

4.1	Arhitektura sistema — komponente i tok podataka	12
6.1	Tok mehanizma objašnjenja rute (<code>explain.Explain</code>)	22
7.1	Pregled entiteta baze podataka (skraćeno, bez svih kolona)	29

Spisak tabela

2.1	Poređenje pristupa rutiranju teretnih vozila	5
4.1	Servisi definisani u <code>docker-compose.yml</code>	14
6.1	Faktor preferencije klase puta (<code>roadClassMultiplier</code>)	24
7.1	Pregled REST API endpoint-a (skraćeno)	27
7.1	Pregled REST API endpoint-a (skraćeno)	28
7.2	Migracije šeme baze podataka	29
7.2	Migracije šeme baze podataka	30
9.1	Broj automatizovanih testova po Go paketu	38
9.1	Broj automatizovanih testova po Go paketu	39
9.2	Vreme izvršavanja nad koridorom Beograd–Novi Sad (38 265 čvorova, 50 705 ivica)	40

Spisak listinga

Listings

4.1	Definicija PostgreSQL/PostGIS servisa u <code>docker-compose.yml</code>	13
5.1	Osmium filter za ekstrakciju podataka relevantnih za teretni saobraćaj . . .	16
6.1	Zahtev za truck-costed rutu sa alternativama (visina/širina/dužina u metrima, masa/osovinsko opterećenje u tonama)	18
6.2	<code>valhalla.Client.RouteAlternates</code> — mapiranje profila vozila u Valhalla <code>costing_options</code> (<code>backend/internal/valhalla/client.go</code>) .	19
6.3	Funkcija bodovanja rute (<code>backend/internal/scoring/scoring.go</code>) — manji rezultat je bolji	20
6.4	Isključivanje nedopustivih ivica i funkcija cene (<code>backend/internal/algorithm/cost.go</code>)	24
6.5	Deljena implementacija Dijkstra i A* pretrage (<code>backend/internal/algorithm/dijkstra.go</code>)	
7.1	Osnovna šema tabela <code>vehicles</code> i <code>trips</code> (<code>00001_initial_schema.sql</code>)	30
7.2	Proračun predloga pauze u <code>TripWorker</code> (<code>backend/internal/worker/trip_worker.go</code>)	32

Biografija

[IME PREZIME] rođen/a je [DATUM ROĐENJA] godine u [MESTO ROĐENJA]. Osnovne akademske studije na Fakultetu tehničkih nauka Univerziteta u Novom Sadu, studijski program [STUDIJSKI PROGRAM], upisao/la je [GODINA UPISA] godine. Tokom studija se posebno interesovao/la za oblasti [OBLASTI INTERESOVANJA – npr. distribuirani sistemi, mobilne aplikacije, geografski informacioni sistemi]. Ovaj diplomski rad predstavlja rezultat rada na projektu razvoja sistema za rutiranje teretnih vozila, izrađenog pod mentorstvom [TITULA I IME MENTORA].

(Ovaj odeljak popunjava kandidat/kinja ličnim podacima pre predaje rada.)